

```
import QtQuick 2.15
import QtQuick.Controls 2.15
import QtQuick.Layouts 1.15
import AStock.Bridge 1.0 as Bridge
import ConsoleUi 1.0
Item {
    id: debugPage
    property var factorService: null
    property var selectedFactorId: ""
    property string validationDate: ""
    property var validationResult: ({} )
    property int refreshCount: 0
    property string lastRefreshLabel: "未刷新"
    property bool refreshFlashActive: false
    property bool validationDateResolved: false
    property bool copyFlashActive: false
    property var recentOperationReports: []
    Timer {
        id: validationDebounceTimer
        interval: 350
        repeat: false
        onTriggered: {
            if (!debugPage.visible) {
                return
            }
            if (selectedFactorId && factorService && /\d{4}-\d{2}-\d{2}$/.test(debugPage.validationDate)) {
                runSingleDayValidation()
                return
            }
            refreshDebugInfo()
        }
    }
}
function fallbackValidationDate() {
    var date = new Date()
    while (date.getDay() === 0 || date.getDay() === 6) {
        date.setDate(date.getDate() - 1)
    }
    return Qt.formatDate(date, "yyyy-MM-dd")
}
function resolveDefaultValidationDate() {
    var fallbackDate = fallbackValidationDate()
    if (factorService && typeof factorService.getLatestAvailableTradeDate === "function") {
        var latestTradeDate = factorService.getLatestAvailableTradeDate()
        if (latestTradeDate && /\d{4}-\d{2}-\d{2}$/.test(latestTradeDate)) {
            console.log("DebugPage 使用数据库最新交易日:", latestTradeDate)
            return latestTradeDate
        }
    }
}
var cleanedDataController = Bridge.CleanedDataController
if (!cleanedDataController) {
```

```
        return fallbackDate
    }
    if (!cleanedDataController.isAvailable && typeof cleanedDataController.initialize === "function") {
        cleanedDataController.initialize()
    }
    if (typeof cleanedDataController.refreshDatasets === "function") {
        cleanedDataController.refreshDatasets()
    }
    if (typeof cleanedDataController.getDataDateRange !== "function") {
        return fallbackDate
    }
    var dateRange = cleanedDataController.getDataDateRange()
    if (dateRange && dateRange.endDate && /^d{4}-d{2}-d{2}$/.test(dateRange.endDate)) {
        console.log("DebugPage 使用最新数据日期:", dateRange.endDate)
        return dateRange.endDate
    }
    return fallbackDate
}
function ensureValidationDateResolved() {
    if (validationDateResolved && validationDate) {
        return
    }
    validationDate = resolveDefaultValidationDate()
    validationDateResolved = true
}
function stringifyValue(value) {
    if (value === undefined || value === null) {
        return ""
    }
    if (typeof value === "string") {
        return value
    }
    try {
        return JSON.stringify(value, null, 2)
    } catch (error) {
        return String(value)
    }
}
function buildSummaryText(result) {
    if (!result || Object.keys(result).length === 0) {
        return qsTr("调试信息将在这里显示")
    }
    var lines = []
    lines.push(qsTr("status: ") + (result.status || ""))
    lines.push(qsTr("count: ") + (result.count !== undefined ? result.count : 0))
    if (result.error) {
        lines.push(qsTr("error: ") + result.error)
    }
    if (result.message) {
        lines.push(qsTr("message: ") + result.message)
    }
}
```

```
}
var diagnostics = result.diagnostics || {}
var diagnosticKeys = Object.keys(diagnostics)
if (diagnosticKeys.length > 0) {
    lines.push("")
    lines.push(qsTr("诊断信息:"))
    for (var diagnosticIndex = 0; diagnosticIndex < diagnosticKeys.length; diagnosticIndex++) {
        var diagnosticKey = diagnosticKeys[diagnosticIndex]
        lines.push(diagnosticKey + ": " + stringifyValue(diagnostics[diagnosticKey]))
    }
}
var stockValues = result.stockValues || {}
var symbols = Object.keys(stockValues)
if (symbols.length > 0) {
    lines.push("")
    lines.push(qsTr("前 10 个样本值:"))
    for (var index = 0; index < Math.min(symbols.length, 10); index++) {
        var symbol = symbols[index]
        lines.push(symbol + ": " + stockValues[symbol])
    }
}
lines.push("")
lines.push(qsTr("完整结果:"))
lines.push(stringifyValue(result))
return lines.join("\n")
}
function buildDiagnosticHint(result) {
    if (!result || Object.keys(result).length === 0) {
        return ""
    }
    if (result.status === "error") {
        return result.error || qsTr("本次验证失败，请先查看 error 字段。")
    }
    var diagnostics = result.diagnostics || {}
    var queriedRows = diagnostics.queriedRows !== undefined ? Number(diagnostics.queriedRows) : -1
    var validRows = diagnostics.validRows !== undefined ? Number(diagnostics.validRows) : -1
    var metric = diagnostics.metric || ""
    var sourceTable = diagnostics.sourceTable || ""
    if (result.count > 0) {
        return qsTr("本次验证成功，已计算出 ") + result.count + qsTr(" 条股票因子值。")
    }
    if (queriedRows > 0 && validRows === 0) {
        return qsTr("已从 ") + sourceTable + qsTr(" 查到 ") + queriedRows
            + qsTr(" 行原始数据， 但指标 ") + metric
            + qsTr(" 全部为 0、负值或无效值，所以最终 count 为 0。")
    }
    if (queriedRows === 0) {
        return qsTr("当前日期没有查到可用于该因子计算的原始数据。")
    }
}
if (result.message) {
```

```

        return result.message
    }
    return qsTr("本次验证没有产出有效股票值，请结合诊断信息继续判断。")
}
function operationTone(report) {
    if (!report || Object.keys(report).length === 0) {
        return "#94A3B8"
    }
    return report.success ? "#22C55E" : "#F87171"
}
function formatOperationHeadline(report) {
    if (!report || Object.keys(report).length === 0) {
        return qsTr("暂无写操作记录")
    }
    var operation = report.operation || "factorOperation"
    var action = operation === "addFactor"
        ? qsTr("新增因子")
        : operation === "updateFactor"
        ? qsTr("更新因子")
        : operation === "deleteFactor"
        ? qsTr("删除因子")
        : operation
    return action + " · " + (report.factorId || qsTr("未指定因子"))
}
function formatOperationDetails(report) {
    if (!report || Object.keys(report).length === 0) {
        return ""
    }
    var parts = []
    parts.push((report.success ? qsTr("成功") : qsTr("失败")) + " / " + (report.stage || "unknown"))
    if (report.message) {
        parts.push(report.message)
    }
    if (report.timestamp) {
        parts.push(report.timestamp)
    }
    return parts.join(" · ")
}
function refreshOperationReports() {
    if (factorService && factorService.recentOperationReports) {
        recentOperationReports = factorService.recentOperationReports || []
    } else {
        recentOperationReports = []
    }
}
function refreshDebugInfo() {
    var now = new Date()
    refreshCount += 1
    lastRefreshLabel = Qt.formatDateTime(now, "yyyy-MM-dd hh:mm:ss") + "." + ("00" + now.getMilliseconds()).slice(-3)
}

```

```
refreshFlashActive = true
refreshFlashTimer.restart()
console.log("DebugPage 刷新调试信息:", refreshCount, lastRefreshLabel)
var lines = []
lines.push(qsTr("刷新次数: ") + refreshCount)
lines.push(qsTr("最后刷新: ") + lastRefreshLabel)
lines.push(qsTr("因子服务: ") + (factorService ? qsTr("已连接") : qsTr("未连接")))
lines.push(qsTr("当前时间: ") + now.toLocaleString())
lines.push(qsTr("选中的因子 ID: ") + (selectedFactorId || qsTr("无")))
lines.push(qsTr("验证日期: ") + validationDate)
if (recentOperationReports && recentOperationReports.length > 0) {
    lines.push("")
    lines.push(qsTr("最近写操作历史:"))
    for (var historyIndex = 0; historyIndex < recentOperationReports.length; historyIndex++) {
        var report = recentOperationReports[historyIndex]
        lines.push("- " + formatOperationHeadline(report))
        lines.push(" " + formatOperationDetails(report))
    }
}
if (validationResult && Object.keys(validationResult).length > 0) {
    lines.push("")
    lines.push(buildSummaryText(validationResult))
}
debugInfo.text = lines.join("\n")
}
function runSingleDayValidation() {
    if (!factorService) {
        validationResult = {
            status: "error",
            error: qsTr("FactorService 未连接")
        }
        refreshDebugInfo()
        return
    }
    if (!selectedFactorId) {
        validationResult = {
            status: "error",
            error: qsTr("请先在因子库选择一个因子")
        }
        refreshDebugInfo()
        return
    }
    var trimmedDate = validationDate ? validationDate.trim() : ""
    if (!(/^\d{4}-\d{2}-\d{2}$/.test(trimmedDate))) {
        validationResult = {
            status: "error",
            error: qsTr("日期格式必须为 yyyy-MM-dd")
        }
        refreshDebugInfo()
        return
    }
}
```

```
}
debugInfo.text = qsTr("正在验证因子 ") + selectedFactorId + qsTr(" 在 ") + trimmedDate + qsTr(" 的单日结果...")
var result = factorService.getFactorValues(selectedFactorId, trimmedDate)
validationResult = result || {
    status: "error",
    error: qsTr("未返回验证结果")
}
refreshDebugInfo()
}
function autoValidateCurrentSelection() {
    ensureValidationDateResolved()
    if (visible && selectedFactorId && factorService) {
        if (!/^\\d{4}-\\d{2}-\\d{2}$/.test(validationDate)) {
            refreshDebugInfo()
            return
        }
        runSingleDayValidation()
        return
    }
    refreshDebugInfo()
}
component DebugActionButton: Rectangle {
    property string label: ""
    property color buttonColor: "#3B82F6"
    signal triggered()
    z: 2
    radius: 6
    color: buttonColor
    implicitWidth: Math.max(96, buttonLabel.implicitWidth + 24)
    implicitHeight: 34
    Text {
        id: buttonLabel
        anchors.centerIn: parent
        text: parent.label
        color: "white"
        font.pixelSize: 14
        font.weight: Font.Medium
    }
    MouseArea {
        anchors.fill: parent
        cursorShape: Qt.PointingHandCursor
        preventStealing: true
        onClicked: parent.triggered()
    }
}
}
Timer {
    id: refreshFlashTimer
    interval: 1200
    repeat: false
    onTriggered: refreshFlashActive = false
}
```

```

}
Timer {
    id: copyFlashTimer
    interval: 1200
    repeat: false
    onTriggered: copyFlashActive = false
}
function copyDebugOutput() {
    var debugText = debugInfo.text ? debugInfo.text.trim() : ""
    if (!debugText) {
        return
    }
    debugInfo.selectAll()
    debugInfo.copy()
    debugInfo.deselect()
    copyFlashActive = true
    copyFlashTimer.restart()
}
Rectangle {
    anchors.fill: parent
    color: "transparent"
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 20
        spacing: 15
        Text {
            text: qsTr("因子调试页面")
            font.pixelSize: 24
            font.bold: true
            color: "white"
        }
        Rectangle {
            Layout.fillWidth: true
            Layout.preferredHeight: 1
            color: "#333"
        }
        Text {
            text: qsTr("选中的因子 ID: ") + (selectedFactorId || qsTr("无"))
            font.pixelSize: 16
            color: "#aaa"
        }
        Text {
            text: qsTr("最后刷新: ") + lastRefreshLabel + qsTr(" | 次数: ") + refreshCount
            font.pixelSize: 13
            color: refreshFlashActive ? "#22C55E" : "#94A3B8"
        }
        Rectangle {
            Layout.fillWidth: true
            radius: 8
            color: validationResult && validationResult.status === "error" ? "#451A1A"

```

```
        : validationResult && validationResult.count > 0 ? "#0F3D2E"
        : "#1E293B"
border.color: validationResult && validationResult.status === "error" ? "#F87171"
        : validationResult && validationResult.count > 0 ? "#22C55E"
        : "#475569"
border.width: 1
visible: !!buildDiagnosticHint(validationResult)
Text {
    anchors.fill: parent
    anchors.margins: 12
    wrapMode: Text.WordWrap
    text: buildDiagnosticHint(validationResult)
    color: validationResult && validationResult.status === "error" ? "#FECACA"
        : validationResult && validationResult.count > 0 ? "#DCFCF7"
        : "#CBD5E1"
    font.pixelSize: 13
}
}
Rectangle {
    Layout.fillWidth: true
    Layout.preferredHeight: 36
    radius: 8
    color: refreshFlashActive ? "#14532D" : "#1F2937"
    border.color: refreshFlashActive ? "#22C55E" : "#334155"
    border.width: 1
    Text {
        anchors.centerIn: parent
        text: refreshFlashActive
            ? qsTr("调试信息已刷新 · ") + lastRefreshLabel
            : qsTr("切换因子、修改日期或执行验证时会自动刷新调试信息")
        color: refreshFlashActive ? "#DCFCF7" : "#94A3B8"
        font.pixelSize: 13
        font.weight: refreshFlashActive ? Font.DemiBold : Font.Normal
    }
}
Rectangle {
    Layout.fillWidth: true
    radius: 8
    color: "#111827"
    border.color: "#334155"
    border.width: 1
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 12
        spacing: 10
        Text {
            text: qsTr("单日单因子验证")
            font.pixelSize: 16
            font.bold: true
            color: "white"
        }
    }
}
```



```

    }
    RowLayout {
        Layout.fillWidth: true
        spacing: 10
        Text {
            text: qsTr("交易日")
            color: "#CBD5E1"
            font.pixelSize: 14
        }
        TextField {
            id: dateField
            Layout.preferredWidth: 180
            text: debugPage.validationDate
            placeholderText: "yyyy-MM-dd"
            color: "white"
            selectByMouse: true
            onTextChanged: debugPage.validationDate = text
            background: Rectangle {
                radius: 6
                color: "#0F172A"
                border.color: parent.activeFocus ? "#3B82F6" : "#334155"
                border.width: 1
            }
        }
        DebugActionButton {
            label: qsTr("单日验证")
            buttonColor: selectedFactorId ? "#2563EB" : "#475569"
            onTriggered: runSingleDayValidation()
        }
        Item { Layout.fillWidth: true }
    }
    Text {
        Layout.fillWidth: true
        wrapMode: Text.WordWrap
        text: validationResult && validationResult.status
            ? qsTr("最近结果:") + validationResult.status
            + (validationResult.count !== undefined ? qsTr(", 数量:") + validationResult.count : ""
)
            + (validationResult.error ? qsTr(", 错误:") + validationResult.error : "")
            : qsTr("说明: 这里调用真实的 FactorService.getFactorValues, 不使用 Debug 页面里的模拟数据
。")
        color: validationResult && validationResult.status === "success" ? "#22C55E"
            : validationResult && validationResult.status === "error" ? "#F87171"
            : "#94A3B8"
        font.pixelSize: 13
    }
    RowLayout {
        Layout.fillWidth: true
        spacing: 10
        Text {

```

```

        text: copyFlashActive ? qsTr("调试输出已复制到剪贴板") : qsTr("可复制当前调试输出，便于排查
或分享结果")
        color: copyFlashActive ? "#22C55E" : "#94A3B8"
        font.pixelSize: 12
    }
    Item { Layout.fillWidth: true }
    DebugActionButton {
        label: copyFlashActive ? qsTr("已复制") : qsTr("复制输出")
        buttonColor: copyFlashActive ? "#16A34A" : (debugInfo.text && debugInfo.text.trim() ? "#0EA5
E9" : "#475569")
        onTriggered: copyDebugOutput()
    }
}
}
}
Rectangle {
    Layout.fillWidth: true
    radius: 8
    color: "#111827"
    border.color: "#334155"
    border.width: 1
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 12
        spacing: 10
        Text {
            text: qsTr("最近因子写操作")
            font.pixelSize: 16
            font.bold: true
            color: "white"
        }
        Text {
            Layout.fillWidth: true
            wrapMode: Text.WordWrap
            text: recentOperationReports.length > 0
                ? qsTr("这里显示 FactorService 最近的新增、更新、删除结果，便于排查失败阶段和重复提交。")
                : qsTr("最近还没有因子写操作记录。")
            color: "#94A3B8"
            font.pixelSize: 12
        }
    }
    Repeater {
        model: recentOperationReports
        delegate: Rectangle {
            required property var modelData
            Layout.fillWidth: true
            radius: 6
            color: "#0F172A"
            border.color: debugPage.operationTone(modelData)
            border.width: 1
            implicitHeight: operationColumn.implicitHeight + 16
        }
    }
}

```

```
Column {  
    id: operationColumn  
    anchors.fill: parent  
    anchors.margins: 8  
    spacing: 4  
    Text {  
        width: parent.width  
        text: debugPage.formatOperationHeadline(modelData)  
        font.pixelSize: 13  
        font.bold: true  
        color: debugPage.operationTone(modelData)  
        wrapMode: Text.WordWrap  
    }  
    Text {  
        width: parent.width  
        text: debugPage.formatOperationDetails(modelData)  
        font.pixelSize: 12  
        color: "#CBD5E1"  
        wrapMode: Text.WordWrap  
    }  
}  
  
} } } } } } } }  
  
ScrollView {  
    Layout.fillWidth: true  
    Layout.fillHeight: true  
    TextArea {  
        id: debugInfo  
        readOnly: true  
        selectByMouse: true  
        text: qsTr("调试信息将在这里显示")  
        background: Rectangle {  
            color: "#1a1a1a"  
            border.color: "#333"  
            border.width: 1  
            radius: 5  
        }  
        color: "#ccc"  
        font.family: "Consolas, 'Courier New', monospace"  
        font.pixelSize: 14  
        padding: 10  
    }  
}  
  
}  
  
Component.onCompleted: {  
    console.log("DebugPage 初始化完成")  
    ensureValidationDateResolved()  
}
```

```
        refreshOperationReports()
        refreshDebugInfo()
    }
    onVisibleChanged: {
        if (visible) {
            ensureValidationDateResolved()
            autoValidateCurrentSelection()
        }
    }
    onSelectedFactorIdChanged: {
        if (selectedFactorId) {
            console.log("DebugPage 接收到新的因子 ID:", selectedFactorId)
        }
        ensureValidationDateResolved()
        autoValidateCurrentSelection()
    }
    onValidationDateChanged: {
        if (visible) {
            validationDebounceTimer.restart()
        } else {
            refreshDebugInfo()
        }
    }
    }
    onFactorServiceChanged: {
        refreshOperationReports()
        if (visible) {
            validationDebounceTimer.restart()
        } else {
            refreshDebugInfo()
        }
    }
    }
    onRecentOperationReportsChanged: refreshDebugInfo()
    Connections {
        target: factorService
        function onRecentOperationReportsChanged() {
            debugPage.refreshOperationReports()
            debugPage.refreshDebugInfo()
        }
    }
    }
}
import QtQuick 2.15
import QtQuick.Controls 2.15
import QtQuick.Layouts 1.15
Item {
    id: root
    property string label: ""
    property var options: []
    property string textRole: "label"
    property string placeholder: "请选择"
    property real fieldWidth: 180
```

```
property real fieldHeight: 36
property int popupMaxHeight: 240
property alias currentIndex: comboBox.currentIndex
property string currentText: optionTextForIndex(comboBox.currentIndex)
property bool enabled: true
signal optionSelected(int index, var option)
implicitWidth: fieldWidth
implicitHeight: contentColumn.implicitHeight
Layout.preferredWidth: fieldWidth
ColumnLayout {
    id: contentColumn
    anchors.fill: parent
    spacing: 4
    Text {
        text: root.label
        font.pixelSize: 12
        color: "#94A3B8"
    }
    ComboBox {
        id: comboBox
        Layout.preferredWidth: root.fieldWidth
        Layout.preferredHeight: root.fieldHeight
        model: root.options
        enabled: root.enabled
        displayText: root.optionTextForIndex(currentIndex)
        onCurrentIndexChanged: {
            if (currentIndex < 0 || currentIndex >= root.options.length) {
                return
            }
            root.optionSelected(currentIndex, root.options[currentIndex])
        }
    }
    background: Rectangle {
        implicitHeight: root.fieldHeight
        radius: 6
        color: "#0F172A"
        border.width: 1
        border.color: "#334155"
        opacity: comboBox.enabled ? 1.0 : 0.5
    }
    contentItem: Text {
        text: comboBox.displayText
        font.pixelSize: 12
        color: comboBox.currentIndex >= 0 ? "#F1F5F9" : "#64748B"
        horizontalAlignment: Text.AlignLeft
        verticalAlignment: Text.AlignVCenter
        leftPadding: 8
        rightPadding: 24
        elide: Text.ElideRight
    }
    indicator: Text {
```

```

        x: comboBox.width - width - 10
        y: comboBox.topPadding + (comboBox.availableHeight - height) / 2
        text: comboBox.popup.visible ? "▲": "▼"
        font.pixelSize: 10
        color: "#94A3B8"
    }
    popup: Popup {
        y: comboBox.height + 4
        width: comboBox.width
        implicitHeight: Math.min(contentItem.implicitHeight + 8, root.popupMaxHeight)
        padding: 4
        contentItem: ListView {
            implicitHeight: contentHeight
            model: comboBox.popup.visible ? comboBox.delegateModel : null
            currentIndex: comboBox.highlightedIndex
            clip: true
            ScrollIndicator.vertical: ScrollIndicator {}
        }
        background: Rectangle {
            color: "#1E293B"
            border.width: 1
            border.color: "#334155"
            radius: 6
        }
    }
}
delegate: ItemDelegate {
    width: comboBox.width - 8
    height: 32
    highlighted: comboBox.highlightedIndex === index
    onClicked: {
        comboBox.currentIndex = index
        comboBox.popup.close()
    }
    contentItem: Text {
        text: root.optionText(modelData)
        color: highlighted ? "#3B82F6": "#F1F5F9"
        font.pixelSize: 12
        verticalAlignment: Text.AlignVCenter
        elide: Text.ElideRight
    }
    background: Rectangle {
        color: highlighted ? "#0F172A": "transparent"
        radius: 4
    }
}
}
}
function optionText(option) {
    if (option === undefined || option === null) {
        return ""
    }
}

```

```
    }
    if (typeof option === "string") {
        return option
    }
    if (textRole && option[textRole] !== undefined && option[textRole] !== null && option[textRole] !== "") {
        return String(option[textRole])
    }
    if (option.label !== undefined && option.label !== null && option.label !== "") {
        return String(option.label)
    }
    if (option.displayText !== undefined && option.displayText !== null && option.displayText !== "") {
        return String(option.displayText)
    }
    if (option.value !== undefined && option.value !== null) {
        return String(option.value)
    }
    return String(option)
}
function optionTextForIndex(index) {
    if (index >= 0 && index < options.length) {
        return optionText(options[index])
    }
    return placeholder
}
}
import QtQuick 2.15
import QtQuick.Layouts 1.15
import ".." as SharedComponents
Item {
    id: root
    property string label: ""
    property string placeholder: "YYYY-MM-DD"
    property alias selectedDate: datePicker.selectedDate
    property bool enabled: true
    property real fieldWidth: 180
    property real fieldHeight: 36
    signal dateChanged(string dateText)
    implicitWidth: fieldWidth
    implicitHeight: contentColumn.implicitHeight
    Layout.preferredWidth: fieldWidth
    ColumnLayout {
        id: contentColumn
        anchors.fill: parent
        spacing: 4
        Text {
            text: root.label
            font.pixelSize: 12
            color: root.enabled ? "#94A3B8" : "#64748B"
        }
        SharedComponents.DatePicker {
```

```
        id: datePicker
        Layout.preferredWidth: root.fieldWidth
        Layout.preferredHeight: root.fieldHeight
        placeholder: root.placeholder
        enabled: root.enabled
        opacity: root.enabled ? 1 : 0.6
        onDateChanged: function(dateText) {
            root.dateChanged(dateText)
        }
    }
}
}
import QtQuick 2.15
import QtQuick.Layouts 1.15
Item {
    id: root
    property string title: ""
    property string value: ""
    property string description: ""
    property string trend: "neutral"
    property color backgroundColor: "#0F172A"
    property color titleColor: "#F1F5F9"
    property color descriptionColor: "#94A3B8"
    property color neutralColor: "#F1F5F9"
    property color upColor: "#EF4444"
    property color downColor: "#10B981"
    property bool hideTrendWhenNeutral: true
    property bool hideTrendForPlaceholder: true
    property int cardHeight: 84
    property int titleSize: 13
    property int valueSize: 18
    property int descriptionSize: 9
    property int trendSize: 14
    Layout.fillWidth: true
    Layout.preferredHeight: cardHeight
    Rectangle {
        anchors.fill: parent
        radius: 8
        color: root.backgroundColor
        ColumnLayout {
            anchors.fill: parent
            anchors.margins: 10
            spacing: 2
            Text {
                text: root.title
                font.pixelSize: root.titleSize
                font.weight: Font.DemiBold
                color: root.titleColor
            }
        }
    }
    Row {
```



```

        spacing: 6
        Text {
            text: root.value
            font.pixelSize: root.valueSize
            font.weight: Font.DemiBold
            color: root.getValueColor()
        }
        Text {
            visible: root.showTrendIndicator()
            text: root.trend === "up" ? "↑" : "↓"
            font.pixelSize: root.trendSize
            color: root.trend === "up" ? root.upColor : root.downColor
        }
    }
    Text {
        text: root.description
        font.pixelSize: root.descriptionSize
        color: root.descriptionColor
    }
}
}
function showTrendIndicator() {
    if (hideTrendWhenNeutral && trend === "neutral") {
        return false
    }
    if (hideTrendForPlaceholder && (value === "--" || value === "N/A")) {
        return false
    }
    return trend === "up" || trend === "down"
}
function getValueColor() {
    if (value === "--" || value === "N/A") return descriptionColor
    if (trend === "up") return upColor
    if (trend === "down") return downColor
    return neutralColor
}
}
import QtQuick 2.15
import QtQuick.Layouts 1.15
import QtQuick.Controls 2.15
import QtCharts 2.15
Rectangle {
    id: root
    property var backtestResult: ({} )
    property var chartDateLabels: []
    property real portfolioAxisMin: 0
    property real portfolioAxisMax: 1
    property real drawdownAxisMin: -1
    property real drawdownAxisMax: 0
    property bool portfolioTooltipVisible: false

```

```
property string portfolioTooltipText: ""
property real portfolioTooltipX: 0
property real portfolioTooltipY: 0
property bool drawdownTooltipVisible: false
property string drawdownTooltipText: ""
property real drawdownTooltipX: 0
property real drawdownTooltipY: 0
property string strategyDisplayName: ""
property bool isBacktesting: false
property alias netValueSeries: netValueSeries
property alias portfolioBoundarySeries: portfolioBoundarySeries
property alias portfolioPeakSeries: portfolioPeakSeries
property alias drawdownSeries: drawdownSeries
property alias drawdownBoundarySeries: drawdownBoundarySeries
property alias drawdownTroughSeries: drawdownTroughSeries
property alias portfolioChartView: portfolioChartView
property alias drawdownChartView: drawdownChartView
property alias detailedResultsDialog: detailedResultsDialog
signal optimizationRequested()
signal exportRequested()
Layout.fillWidth: true
implicitHeight: resultContent.implicitHeight + 32
Layout.preferredHeight: implicitHeight
radius: 12
color: "#1E293B"
onBacktestResultChanged: refreshTimeSeriesCharts()
ColumnLayout {
    id: resultContent
    anchors.fill: parent
    anchors.margins: 16
    spacing: 10
    RowLayout {
        spacing: 8
        Text {
            text: "    策略回测结果"
            font.pixelSize: 16
            font.weight: Font.DemiBold
            color: "#F1F5F9"
        }
        Item { Layout.fillWidth: true }
        Text {
            text: root.resultStatusText()
            font.pixelSize: 12
            color: root.resultStatusColor()
        }
    }
}
GridLayout {
    Layout.fillWidth: true
    Layout.preferredHeight: 184
    columns: 3
```

```

        columnSpacing: 12
        rowSpacing: 12
        BacktestMetricCard {
            title: "总收益"
            value: root.formatPercent(root.performanceSummary().totalReturn, 1)
            description: "Total Return"
            trend: root.performanceSummary().totalReturn > 0 ? "up" : (root.performanceSummary().totalReturn < 0 ? "
down" : "neutral")
        }
        BacktestMetricCard {
            title: "年化收益"
            value: root.formatPercent(root.performanceSummary().annualizedReturn, 1)
            description: "Annual Return"
            trend: root.performanceSummary().annualizedReturn > 0 ? "up" : (root.performanceSummary().annualizedRetu
rn < 0 ? "down" : "neutral")
        }
        BacktestMetricCard {
            title: "夏普比率"
            value: root.formatNumber(root.performanceSummary().sharpeRatio, 2)
            description: "Sharpe Ratio"
            trend: root.performanceSummary().sharpeRatio > 0 ? "up" : (root.performanceSummary().sharpeRatio < 0 ? "
down" : "neutral")
        }
        BacktestMetricCard {
            title: "最大回撤"
            value: root.formatPercent(root.performanceSummary().maxDrawdown, 1)
            description: "Max Drawdown"
            trend: "down"
        }
        BacktestMetricCard {
            title: "胜率"
            value: root.formatPercent(root.performanceSummary().winRate, 1)
            description: "Win Rate"
            trend: root.performanceSummary().winRate > 0.5 ? "up" : (root.performanceSummary().winRate < 0.5 ? "down
" : "neutral")
        }
        BacktestMetricCard {
            title: "盈亏比"
            value: root.formatNumber(root.performanceSummary().profitFactor, 2)
            description: "Profit/Loss Ratio"
            trend: root.performanceSummary().profitFactor > 1 ? "up" : (root.performanceSummary().profitFactor < 1 ?
"down" : "neutral")
        }
    }
    Rectangle {
        Layout.fillWidth: true
        Layout.preferredHeight: root.hasPortfolioContext() ? 92 : 0
        radius: 8
        color: "#0F172A"
        visible: root.hasPortfolioContext()
    }

```

```
ColumnLayout {
    anchors.fill: parent
    anchors.margins: 12
    spacing: 8
    Text {
        text: "        组合回测上下文"
        font.pixelSize: 14
        font.weight: Font.DemiBold
        color: "#F1F5F9"
    }
    GridLayout {
        Layout.fillWidth: true
        columns: 2
        columnSpacing: 16
        rowSpacing: 8
        BacktestStatItem {
            label: "执行类型"
            value: root.backtestResult.executionKindLabel || "--"
        }
        BacktestStatItem {
            label: "因子数量"
            value: root.formatInteger(root.factorOverlaySelectedFactors().length)
        }
        BacktestStatItem {
            label: "数据源"
            value: root.backtestResult.dataSourceModeLabel || "--"
        }
        BacktestStatItem {
            label: "因子列表"
            value: root.factorOverlayFactorIdsText()
        }
    }
}
}
}
Rectangle {
    Layout.fillWidth: true
    Layout.preferredHeight: root.hasExecutionRiskContext() ? 92 : 0
    radius: 8
    color: "#0F172A"
    visible: root.hasExecutionRiskContext()
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 12
        spacing: 8
        Text {
            text: "        本次执行参数"
            font.pixelSize: 14
            font.weight: Font.DemiBold
            color: "#F1F5F9"
        }
    }
}
```

```
        GridLayout {
            Layout.fillWidth: true
            columns: 2
            columnSpacing: 16
            rowSpacing: 8
            BacktestStatItem {
                label: "止损"
                value: root.formatPercent(root.ruleProfile().stopLossPercent, 1)
            }
            BacktestStatItem {
                label: "止盈"
                value: root.formatPercent(root.ruleProfile().takeProfitPercent, 1)
            }
            BacktestStatItem {
                label: "最大回撤限制"
                value: root.formatPercent(root.ruleProfile().maxDrawdownLimit, 1)
            }
            BacktestStatItem {
                label: "调仓周期"
                value: root.formatInteger(root.executionPolicy().rebalanceDays)
            }
        }
    }
}
Rectangle {
    Layout.fillWidth: true
    Layout.preferredHeight: root.hasRuleTemplateSummary() ? 132 : 0
    radius: 8
    color: "#0F172A"
    visible: root.hasRuleTemplateSummary()
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 12
        spacing: 8
        RowLayout {
            Layout.fillWidth: true
            Text {
                text: "    规则模板命中"
                font.pixelSize: 14
                font.weight: Font.DemiBold
                color: "#F1F5F9"
            }
            Item { Layout.fillWidth: true }
            Text {
                text: (root.backtestResult.ruleTemplateSummary || {}).statusText || ""
                font.pixelSize: 11
                color: "#94A3B8"
            }
        }
    }
}
GridLayout {
```

```

        Layout.fillWidth: true
        columns: 2
        columnSpacing: 16
        rowSpacing: 8
        BacktestStatItem {
            label: "规则文件"
            value: (root.backtestResult.ruleTemplateSummary || {}).templateFileName || "--"
        }
        BacktestStatItem {
            label: "命名空间"
            value: (root.backtestResult.ruleTemplateSummary || {}).templateNamespace || "--"
        }
        BacktestStatItem {
            label: "所属规则组"
            value: root.ruleTemplateGroupText(root.backtestResult.ruleTemplateSummary || {})
        }
        BacktestStatItem {
            label: "组合语义"
            value: root.ruleTemplateGroupLogicText(root.backtestResult.ruleTemplateSummary || {})
        }
        BacktestStatItem {
            label: "开仓阻断"
            value: root.formatInteger((root.backtestResult.ruleTemplateSummary || {}).entryBlockCount)
        }
        BacktestStatItem {
            label: "规则退出"
            value: root.formatInteger((root.backtestResult.ruleTemplateSummary || {}).forcedExitCount)
        }
    }
}
}
Rectangle {
    Layout.fillWidth: true
    Layout.preferredHeight: 104
    radius: 8
    color: "#0F172A"
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 12
        spacing: 8
        Text {
            text: "    交易统计"
            font.pixelSize: 14
            font.weight: Font.DemiBold
            color: "#F1F5F9"
        }
    }
    GridLayout {
        Layout.fillWidth: true
        Layout.fillHeight: true
        columns: 3
    }
}

```

```
        columnSpacing: 16
        rowSpacing: 8
        BacktestStatItem {
            label: "总交易次数"
            value: (root.tradeSummary().totalTrades || 0)
        }
        BacktestStatItem {
            label: "盈利交易"
            value: (root.tradeSummary().winningTrades || 0)
            valueColor: "#EF4444"
        }
        BacktestStatItem {
            label: "亏损交易"
            value: (root.tradeSummary().losingTrades || 0)
            valueColor: "#10B981"
        }
        BacktestStatItem {
            label: "平均盈利"
            value: root.formatPercent(root.performanceSummary().averageWin, 2)
            valueColor: "#EF4444"
        }
        BacktestStatItem {
            label: "平均亏损"
            value: root.formatPercent(root.performanceSummary().averageLoss, 2)
            valueColor: "#10B981"
        }
        BacktestStatItem {
            label: "交易天数"
            value: ((root.backtestResult.timeSeries || {}).dates || []).length
        }
    }
}
Rectangle {
    Layout.fillWidth: true
    Layout.preferredHeight: 228
    radius: 8
    color: "#0F172A"
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 12
        spacing: 10
        RowLayout {
            Layout.fillWidth: true
            Text {
                text: "      净值曲线"
                font.pixelSize: 14
                font.weight: Font.DemiBold
                color: "#F1F5F9"
            }
        }
    }
}
```

```
Item { Layout.fillWidth: true }
Text {
    text: root.chartDateLabels.length > 0
        ? (root.chartDateLabels[0] + " ~ " + root.chartDateLabels[root.chartDateLabels.length - 1])
        : "等待回测结果"
    font.pixelSize: 11
    color: "#94A3B8"
}
}
}
ChartView {
    id: portfolioChartView
    Layout.fillWidth: true
    Layout.fillHeight: true
    antialiasing: true
    legend.visible: false
    backgroundColor: "transparent"
    plotAreaColor: "transparent"
    ValueAxis {
        id: portfolioXAxis
        min: 0
        max: Math.max(1, netValueSeries.count > 0 ? netValueSeries.count - 1 : 1)
        tickCount: Math.min(6, Math.max(2, netValueSeries.count > 1 ? 6 : 2))
        labelsColor: "#64748B"
        gridLineColor: "#1E293B"
        lineVisible: false
    }
    ValueAxis {
        id: portfolioYAxis
        min: root.portfolioAxisMin
        max: root.portfolioAxisMax
        tickCount: 5
        labelsColor: "#94A3B8"
        gridLineColor: "#1E293B"
        labelFormat: "%.0f"
    }
    LineSeries {
        id: netValueSeries
        name: "净值"
        color: "#38BDF8"
        width: 2
        axisX: portfolioXAxis
        axisY: portfolioYAxis
    }
    LineSeries {
        id: benchmarkNetValueSeries
        name: "基准(沪深300)"
        color: "#94A3B8"
        width: 1
        style: Qt.DashLine
        axisX: portfolioXAxis
    }
}
```



```

        axisY: portfolioYAxis
    }
    ScatterSeries {
        id: portfolioBoundarySeries
        axisX: portfolioXAxis
        axisY: portfolioYAxis
        color: "#E2E8F0"
        borderColor: "#0F172A"
        markerSize: 9
    }
    ScatterSeries {
        id: portfolioPeakSeries
        axisX: portfolioXAxis
        axisY: portfolioYAxis
        color: "#10B981"
        borderColor: "#0F172A"
        markerSize: 10
    }
    MouseArea {
        anchors.fill: parent
        hoverEnabled: true
        onExited: root.portfolioTooltipVisible = false
        onPositionChanged: function(mouse) {
            root.updateChartTooltip(portfolioChartView, netValueSeries, mouse.x, mouse.y, false)
        }
    }
    Rectangle {
        visible: root.portfolioTooltipVisible
        x: Math.min(Math.max(0, root.portfolioTooltipX), portfolioChartView.width - width)
        y: Math.min(Math.max(0, root.portfolioTooltipY), portfolioChartView.height - height)
        width: portfolioTooltipLabel.implicitWidth + 16
        height: portfolioTooltipLabel.implicitHeight + 10
        radius: 6
        color: "#020617"
        border.width: 1
        border.color: "#334155"
        opacity: 0.96
        Text {
            id: portfolioTooltipLabel
            anchors.centerIn: parent
            text: root.portfolioTooltipText
            font.pixelSize: 11
            color: "#F8FAFC"
        }
    }
}
RowLayout {
    Layout.fillWidth: true
    Text {
        text: root.chartDateLabels.length > 0 ? root.chartDateLabels[0] : "--"
    }
}

```

```

        font.pixelSize: 11
        color: "#64748B"
    }
    Item { Layout.fillWidth: true }
    Text {
        text: root.chartDateLabels.length > 2 ? root.chartDateLabels[Math.floor((root.chartDateLabels.le
ngth - 1) / 2)] : ""
        font.pixelSize: 11
        color: "#64748B"
    }
    Item { Layout.fillWidth: true }
    Text {
        text: root.chartDateLabels.length > 0 ? root.chartDateLabels[root.chartDateLabels.length - 1] :
"--"
        font.pixelSize: 11
        color: "#64748B"
    }
}
RowLayout {
    Layout.fillWidth: true
    spacing: 12
    Text {
        text: "● 起点/终点"
        font.pixelSize: 11
        color: "#E2E8F0"
    }
    Text {
        text: "● 峰值"
        font.pixelSize: 11
        color: "#10B981"
    }
    Item { Layout.fillWidth: true }
    Text {
        text: "悬浮查看当日净值"
        font.pixelSize: 11
        color: "#64748B"
    }
}
}
}
Rectangle {
    Layout.fillWidth: true
    Layout.preferredHeight: 196
    radius: 8
    color: "#0F172A"
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: 12
        spacing: 10
        RowLayout {

```

```
Layout.fillWidth: true
Text {
    text: "    回撤曲线"
    font.pixelSize: 14
    font.weight: Font.DemiBold
    color: "#F1F5F9"
}
Item { Layout.fillWidth: true }
Text {
    text: netValueSeries.count > 0 ? ("数据点: " + netValueSeries.count) : "等待回测结果"
    font.pixelSize: 11
    color: "#94A3B8"
}
}
}
ChartView {
    id: drawdownChartView
    Layout.fillWidth: true
    Layout.fillHeight: true
    antialiasing: true
    legend.visible: false
    backgroundColor: "transparent"
    plotAreaColor: "transparent"
    ValueAxis {
        id: drawdownXAxis
        min: 0
        max: Math.max(1, drawdownSeries.count > 0 ? drawdownSeries.count - 1 : 1)
        tickCount: Math.min(6, Math.max(2, drawdownSeries.count > 1 ? 6 : 2))
        labelsColor: "#64748B"
        gridLineColor: "#1E293B"
        lineVisible: false
    }
    ValueAxis {
        id: drawdownYAxis
        min: root.drawdownAxisMin
        max: root.drawdownAxisMax
        tickCount: 5
        labelsColor: "#94A3B8"
        gridLineColor: "#1E293B"
        labelFormat: "%.2f"
    }
}
LineSeries {
    id: drawdownSeries
    name: "回撤"
    color: "#F59E0B"
    width: 2
    axisX: drawdownXAxis
    axisY: drawdownYAxis
}
LineSeries {
    id: benchmarkDrawdownSeries
```

```
        name: "基准回撤"
        color: "#94A3B8"
        width: 1
        style: Qt.DashLine
        axisX: drawdownXAxis
        axisY: drawdownYAxis
    }
    ScatterSeries {
        id: drawdownBoundarySeries
        axisX: drawdownXAxis
        axisY: drawdownYAxis
        color: "#E2E8F0"
        borderColor: "#0F172A"
        markerSize: 9
    }
    ScatterSeries {
        id: drawdownTroughSeries
        axisX: drawdownXAxis
        axisY: drawdownYAxis
        color: "#EF4444"
        borderColor: "#0F172A"
        markerSize: 10
    }
    MouseArea {
        anchors.fill: parent
        hoverEnabled: true
        onExited: root.drawdownTooltipVisible = false
        onPositionChanged: function(mouse) {
            root.updateChartTooltip(drawdownChartView, drawdownSeries, mouse.x, mouse.y, true)
        }
    }
    Rectangle {
        visible: root.drawdownTooltipVisible
        x: Math.min(Math.max(0, root.drawdownTooltipX), drawdownChartView.width - width)
        y: Math.min(Math.max(0, root.drawdownTooltipY), drawdownChartView.height - height)
        width: drawdownTooltipLabel.implicitWidth + 16
        height: drawdownTooltipLabel.implicitHeight + 10
        radius: 6
        color: "#020617"
        border.width: 1
        border.color: "#334155"
        opacity: 0.96
        Text {
            id: drawdownTooltipLabel
            anchors.centerIn: parent
            text: root.drawdownTooltipText
            font.pixelSize: 11
            color: "#F8FAFC"
        }
    }
}
```

```
    }
    RowLayout {
        Layout.fillWidth: true
        spacing: 12
        Text {
            text: "● 起点/终点"
            font.pixelSize: 11
            color: "#E2E8F0"
        }
        Text {
            text: "● 最大回撤"
            font.pixelSize: 11
            color: "#EF4444"
        }
        Item { Layout.fillWidth: true }
        Text {
            text: "悬浮查看当日回撤"
            font.pixelSize: 11
            color: "#64748B"
        }
    }
}
}
RowLayout {
    Layout.fillWidth: true
    Layout.preferredHeight: 40
    spacing: 12
    Rectangle {
        Layout.preferredWidth: 120
        Layout.fillHeight: true
        radius: 6
        color: "#334155"
        Row {
            anchors.centerIn: parent
            spacing: 6
            Text {
                text: " "
                font.pixelSize: 12
                color: "#F1F5F9"
            }
            Text {
                text: "详细结果"
                font.pixelSize: 12
                color: "#F1F5F9"
            }
        }
    }
}
MouseArea {
    anchors.fill: parent
    cursorShape: Qt.PointingHandCursor
    enabled: root.hasBacktestResult()
```

```
        onClicked: {
            if (root.hasBacktestResult()) {
                detailedResultsDialog.open()
            }
        }
    }
}
Rectangle {
    Layout.preferredWidth: 100
    Layout.fillHeight: true
    radius: 6
    color: "#334155"
    Row {
        anchors.centerIn: parent
        spacing: 6
        Text {
            text: "    "
            font.pixelSize: 12
            color: "#F1F5F9"
        }
        Text {
            text: "优化"
            font.pixelSize: 12
            color: "#F1F5F9"
        }
    }
}
MouseArea {
    anchors.fill: parent
    cursorShape: Qt.PointingHandCursor
    onClicked: root.optimizationRequested()
}
}
Item { Layout.fillWidth: true }
Rectangle {
    Layout.preferredWidth: 100
    Layout.fillHeight: true
    radius: 6
    color: "#3B82F6"
    enabled: root.backtestResult && Object.keys(root.backtestResult).length > 0
    opacity: enabled ? 1.0 : 0.5
    Row {
        anchors.centerIn: parent
        spacing: 6
        Text {
            text: "    "
            font.pixelSize: 12
            color: "white"
        }
        Text {
            text: "导出"
        }
    }
}
```

```
#include "registerQmlTypes.hpp"
#include "foundation/log/logging.hpp"
#include <QQmlEngine>
#include <QStringList>
#include <QTimer>
#include "PreviewDataModel.h"
#include "DataFetchController.h" // 添加DataFetchController头文件
#include "FactorViewModel.h" // 因子视图模型（只负责视图）
#include "FactorDebugController.h" // 新增：因子调试控制器
#include "FactorMetaService.h" // 新增：因子元数据服务
#include "CleanedDataController.h" // 新增：清洗后数据控制器
#include "CandleDataModel.h"
#include "CrosshairManager.h"
#include "MarketDataBridge.h"
#include "StrategyBridge.h"
#include "StrategyTypeContract.h"
#include "FactorService.h"
#include "FactorBacktestBridge.h"
#include "StrategyBacktestBridge.h"
#include "TradingBridges.h"
#include "TradingConnectionConfigService.h"
#include "TradingRuntimeStatusService.h"
#include "TradingMarketCalendarService.h"
#include "RiskConfigService.h"
#include "TradingFormPanelHelper.h"
#include "UiLifecycleCoordinator.h"
#include "StrategyPerformanceModel.h"
#include "SymbolSearchModel.h"
#include "RuleTemplateDetailHelper.h"
#include "RuleTemplateSuggestionService.h"
#include "DataCleaningServiceRefactored.h"
#include "StrategyRuleStatsBridge.h"
#include "ParameterTuningBridge.h"
namespace wang{
void registerQmlTypes()
{
    static const char* url = "AStock.Bridge";
    // 预览数据模型 - 专为预览窗口设计
    qmlRegisterType<PreviewDataModel>(url, 1, 0, "PreviewDataModel");
    // DataFetchController - 用于数据获取和清洗，遵循不在QML中操作数据的原则
    qmlRegisterType<DataFetchController>(url, 1, 0, "DataFetchController");
    // FactorViewModel - 因子视图模型（只负责视图更新）
    qmlRegisterType<FactorViewModel>(url, 1, 0, "FactorViewModel");
    // FactorDebugController - 因子调试控制器
    qmlRegisterType<FactorDebugController>(url, 1, 0, "FactorDebugController");
    // FactorMetaService - 因子元数据服务（单例模式）
    qmlRegisterSingletonType<FactorMetaService>(
        url, 1, 0, "FactorMetaService",
        [] (QQmlEngine* engine, QJSEngine* scriptEngine) -> QObject* {
            Q_UNUSED(engine)
```

```

    Q_UNUSED(scriptEngine)
    auto* service = new FactorMetaService();
    service->initialize();
    return service;
}
);
// CleanedDataController - 清洗后数据控制器 (单例模式)
qmlRegisterSingletonType<ui::bridge::CleanedDataController>(
    url, 1, 0, "CleanedDataController",
    [](QQmlEngine* engine, QJSEngine* scriptEngine) -> QObject* {
        Q_UNUSED(engine)
        Q_UNUSED(scriptEngine)
        auto* controller = new ui::bridge::CleanedDataController();
        // 同步初始化: 只读缓存元数据索引, 不加载数据文件, 不会阻塞 UI
        controller->initialize();
        return controller;
    }
);
// 共享 CandleDataModel 实例
auto* sharedModel = new bridge::CandleDataModel();
qmlRegisterSingletonType<bridge::CandleDataModel>(
    url, 1, 0, "CandleDataModel",
    [sharedModel](QQmlEngine*, QJSEngine*) -> QObject* {
        return sharedModel;
    }
);
// 统一的行情桥接层 — 同时注册到两个 QML 名字, 兼容现有 QML 代码
auto* marketBridge = new bridge::MarketDataBridge();
marketBridge->setModel(sharedModel);
marketBridge->initializeAsync();
qmlRegisterSingletonInstance<bridge::MarketDataBridge>(
    url, 1, 0, "MarketDataBridge", marketBridge);
qmlRegisterSingletonInstance<bridge::MarketDataBridge>(
    url, 1, 0, "StockDataLoader", marketBridge);
// CrosshairManager — 十字光标状态 (单例, 引擎不接管生命周期)
qmlRegisterSingletonInstance<bridge::CrosshairManager>(
    url, 1, 0, "CrosshairManager",
    &bridge::CrosshairManager::instance()
);
qmlRegisterSingletonType<StrategyBridge>(
    url, 1, 0, "StrategyBridge",
    [](QQmlEngine*, QJSEngine*) -> QObject* {
        INTERNAL_INFO_STREAM << "[QML] StrategyBridge 单例已创建";
        auto* bridge = new StrategyBridge();
        return bridge;
    }
);
// StrategyTypes — 策略类型枚举契约 (不可创建; QML 仅引用其枚举值)
qmlRegisterUncreatableType<StrategyTypeContract>(
    url, 1, 0, "StrategyTypes",
    QStringLiteral("策略类型枚举契约, 仅供 QML 引用枚举值, 禁止实例化"));

```



```
// FactorService - 因子服务桥接层 (单例模式)
// instance() 内部已同步调用 initialize(), 无需 QTimer 延迟
qmlRegisterSingletonType<FactorService>(
    url, 1, 0, "FactorService",
    [](QQmlEngine* engine, QJSEngine* scriptEngine) -> QObject* {
        Q_UNUSED(engine)
        Q_UNUSED(scriptEngine)
        auto* service = FactorService::instance();
        return service;
    }
);
// FactorBacktestController - 因子回测控制器 (QML 内联组件)
qmlRegisterType<FactorBacktestBridge>(
    url, 1, 0, "FactorBacktestController");
// StrategyBacktestController - 策略回测控制器 (QML 内联组件)
qmlRegisterType<StrategyBacktestBridge>(
    url, 1, 0, "StrategyBacktestController");
// —— 交易系统桥接层 (单例) ——
qmlRegisterSingletonType<bridge::TradeExecutionBridge>(
    url, 1, 0, "TradeExecutionBridge",
    [](QQmlEngine*, QJSEngine*) -> QObject* {
        return new bridge::TradeExecutionBridge();
    });
qmlRegisterSingletonType<bridge::PositionAccountBridge>(
    url, 1, 0, "PositionAccountBridge",
    [](QQmlEngine*, QJSEngine*) -> QObject* {
        return new bridge::PositionAccountBridge();
    });
qmlRegisterSingletonType<bridge::RiskControlBridge>(
    url, 1, 0, "RiskControlBridge",
    [](QQmlEngine*, QJSEngine*) -> QObject* {
        return new bridge::RiskControlBridge();
    });
// —— 新增桥接类型 (单例) ——
qmlRegisterSingletonType<bridge::TradingConnectionConfigService>(
    url, 1, 0, "TradingConnectionConfigService",
    [](QQmlEngine*, QJSEngine*) -> QObject* {
        return bridge::TradingConnectionConfigService::instance();
    });
qmlRegisterSingletonType<bridge::TradingRuntimeStatusService>(
    url, 1, 0, "TradingRuntimeStatusService",
    [](QQmlEngine*, QJSEngine*) -> QObject* {
        return new bridge::TradingRuntimeStatusService();
    });
qmlRegisterSingletonType<bridge::TradingMarketCalendarService>(
    url, 1, 0, "TradingMarketCalendarService",
    [](QQmlEngine*, QJSEngine*) -> QObject* {
        return new bridge::TradingMarketCalendarService();
    });
qmlRegisterSingletonType<bridge::RiskConfigService>(
```

```
url, 1, 0, "RiskConfigService",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    return new bridge::RiskConfigService();
});
qmlRegisterSingletonType<bridge::TradingFormPanelHelper>(
url, 1, 0, "TradingFormPanelHelper",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    return new bridge::TradingFormPanelHelper();
});
qmlRegisterSingletonType<bridge::UiLifecycleCoordinator>(
url, 1, 0, "UiLifecycleCoordinator",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    return new bridge::UiLifecycleCoordinator();
});
qmlRegisterType<StrategyPerformanceModel>(
url, 1, 0, "StrategyPerformanceModel");
qmlRegisterType<SymbolSearchModel>(
url, 1, 0, "SymbolSearchModel");
// QML 侧按 singleton 用法直接调 RuleTemplateDetailHelper.describeBinding(),
// 必须注册为 singleton (qmlRegisterType 会导致对类型名调方法 → TypeError)
qmlRegisterSingletonType<RuleTemplateDetailHelper>(
url, 1, 0, "RuleTemplateDetailHelper",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    return new RuleTemplateDetailHelper();
});
qmlRegisterSingletonType<RuleTemplateSuggestionService>(
url, 1, 0, "RuleTemplateSuggestionService",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    return new RuleTemplateSuggestionService();
});
// DataCleaningServiceRefactored — 数据清洗服务
qmlRegisterSingletonType<DataCleaningServiceRefactored>(
url, 1, 0, "DataCleaningServiceRefactored",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    auto* svc = new DataCleaningServiceRefactored();
    svc->initialize();
    return svc;
});
// StrategyRuleStatsBridge — 策略规则统计桥接
qmlRegisterSingletonType<StrategyRuleStatsBridge>(
url, 1, 0, "StrategyRuleStatsBridge",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    return new StrategyRuleStatsBridge();
});
// ParameterTuningBridge — 参数自动调优桥接 (单例)
qmlRegisterSingletonType<ParameterTuningBridge>(
url, 1, 0, "ParameterTuningBridge",
[]((QQmlEngine*, QJSEngine*) -> QObject* {
    return new ParameterTuningBridge();
});
```

```

    }
}
#include "UiLifecycleCoordinator.h"
#include "foundation/log/logging.hpp"
namespace bridge {
UiLifecycleCoordinator::UiLifecycleCoordinator(QObject* parent)
: QObject(parent) {}
void UiLifecycleCoordinator::activateTradingPage() {
    m_tradingPageActivated = true;
    emit tradingPageActivated();
}
void UiLifecycleCoordinator::activateStrategyLibraryPage() {
    INTERNAL_INFO_STREAM << "[Lifecycle] 激活策略库页面";
    m_strategyLibraryPageActivated = true;
    emit strategyLibraryPageActivated();
    INTERNAL_INFO_STREAM << "[Lifecycle] 激活策略库页面 完成";
}
} // namespace bridge
#include "TradingRuntimeStatusService.h"
#include "../engine/include/GmSessionEngine.h"
#include "../domain/strategy/include/StrategyManager.h"
#include <QDebug>
#include <QTimer>
namespace bridge {
TradingRuntimeStatusService::TradingRuntimeStatusService(QObject* parent)
: QObject(parent) {}
QVariantMap TradingRuntimeStatusService::sessionSnapshotForStrategy(const QString& strategyId) {
    QVariantMap snapshot = buildTradingSystemSnapshot();
    snapshot["strategyId"] = strategyId;
    snapshot["sessionId"] = QStringLiteral("strategy_") + strategyId;
    if (engine::GmSessionEngine::instance().initialized()) {
        snapshot["initialized"] = true;
        snapshot["connected"] = true;
        auto* e = domain::strategy::StrategyManager::instance().get(strategyId.toStdString());
        snapshot["state"] = (e && e->isLiveLoopRunning()) ? "Running" : "Ready";
        snapshot["stateLabel"] = (e && e->isLiveLoopRunning()) ? QStringLiteral("运行中") : QStringLiteral("已创建");
    }
    return snapshot;
}
QVariantMap TradingRuntimeStatusService::sessionSnapshotForAccount(const QString& accountId) {
    QVariantMap snapshot = buildTradingSystemSnapshot();
    snapshot["accountId"] = accountId;
    snapshot["sessionId"] = QStringLiteral("account_") + accountId;
    if (engine::GmSessionEngine::instance().initialized()) {
        snapshot["initialized"] = true;
        snapshot["connected"] = true;
        snapshot["state"] = "Ready";
        snapshot["stateLabel"] = QStringLiteral("已创建");
    }
    return snapshot;
}

```

```
}
void TradingRuntimeStatusService::refresh() {
    m_sessionSnapshots.clear();
    if (engine::GmSessionEngine::instance().initialized()) {
        QVariantMap sysSnapshot = buildTradingSystemSnapshot();
        sysSnapshot["sessionId"] = "trading_system";
        m_sessionSnapshots.append(sysSnapshot);
    }
    emit sessionSnapshotsChanged();
}
void TradingRuntimeStatusService::refreshAsync() {
    QTimer::singleShot(0, this, [this]() { refresh(); });
}
QVariantList TradingRuntimeStatusService::sessionSnapshots() const {
    return m_sessionSnapshots;
}
QVariantMap TradingRuntimeStatusService::buildDefaultSnapshot(const QString& sessionId) const {
    QVariantMap snap;
    snap["sessionId"] = sessionId;
    snap["state"] = "Created";
    snap["stateLabel"] = QStringLiteral("已创建");
    snap["connected"] = false;
    snap["initialized"] = false;
    snap["lastError"] = QString();
    snap["subscriptions"] = QVariantList();
    snap["hasError"] = false;
    return snap;
}
QVariantMap TradingRuntimeStatusService::buildTradingSystemSnapshot() const {
    QVariantMap snap;
    snap["state"] = "Created";
    snap["stateLabel"] = QStringLiteral("已创建");
    snap["connected"] = false;
    snap["initialized"] = false;
    snap["lastError"] = QString();
    snap["subscriptions"] = QVariantList();
    snap["hasError"] = false;
    if (engine::GmSessionEngine::instance().initialized()) {
        snap["state"] = "Ready";
        snap["stateLabel"] = QStringLiteral("已创建");
        snap["connected"] = true;
        snap["initialized"] = true;
    }
    return snap;
}
TradingRuntimeStatusService* TradingRuntimeStatusService::instance() {
    static TradingRuntimeStatusService s;
    return &s;
}
} // namespace bridge
```

```
#include "TradingMarketCalendarService.h"
#include <QDate>
#include <QTime>
namespace bridge {
TradingMarketCalendarService::TradingMarketCalendarService(QObject* parent)
: QObject(parent) {
    refreshCalendar();
}
QVariantMap TradingMarketCalendarService::currentSessionSnapshot() const {
    return m_sessionSnapshot;
}
bool TradingMarketCalendarService::isTradingSessionOpen() const {
    return m_sessionSnapshot.value("sessionOpen").toBool();
}
bool TradingMarketCalendarService::isHolidayAware() const {
    return m_sessionSnapshot.value("holidayAware").toBool();
}
void TradingMarketCalendarService::refresh() {
    refreshCalendar();
}
void TradingMarketCalendarService::refreshCalendar() {
    QVariantMap snapshot;
    const QDate today = QDate::currentDate();
    const QTime now = QTime::currentTime();
    // 交易日历感知 (MVP: 工作日检测, 未对接真实假期数据)
    snapshot["holidayAware"] = false;
    snapshot["error"] = QString();
    snapshot["sourceLabel"] = QStringLiteral("内置日历 (工作日检测)");
    // 最新的已确认交易日 (简化: 最近一个工作日)
    QDate lastTrade = today;
    if (today.dayOfWeek() == 7) lastTrade = today.addDays(-1); // 周日→周五
    else if (today.dayOfWeek() == 6) lastTrade = today.addDays(-1); // 周六→周五
    snapshot["latestClosedTradeDate"] = lastTrade.toString("yyyy-MM-dd");
    // 交易时段判断 (中国A股市场时段)
    bool weekend = (today.dayOfWeek() == 6 || today.dayOfWeek() == 7);
    const QTime morningOpen(9, 15);
    const QTime morningClose(11, 30);
    const QTime afternoonOpen(13, 0);
    const QTime afternoonClose(15, 0);
    const QTime postClose(15, 30);
    QString phaseLabel;
    bool sessionOpen = false;
    if (weekend) {
        phaseLabel = QStringLiteral("周末休市");
    } else if (now < morningOpen) {
        phaseLabel = QStringLiteral("等待开盘");
    } else if (now >= morningOpen && now <= morningClose) {
        phaseLabel = QStringLiteral("早盘交易中");
        sessionOpen = true;
    } else if (now > morningClose && now < afternoonOpen) {
```

```

        phaseLabel = QStringLiteral("午间休市");
    } else if (now >= afternoonOpen && now <= afternoonClose) {
        phaseLabel = QStringLiteral("午盘交易中");
        sessionOpen = true;
    } else if (now > afternoonClose && now <= postClose) {
        phaseLabel = QStringLiteral("盘后处理");
    } else {
        phaseLabel = QStringLiteral("已收盘");
    }
    snapshot["sessionPhaseLabel"] = phaseLabel;
    snapshot["sessionOpen"] = sessionOpen;
    m_sessionSnapshot = snapshot;
    emit currentSessionSnapshotChanged();
}
TradingMarketCalendarService* TradingMarketCalendarService::instance() {
    static TradingMarketCalendarService s_instance;
    return &s_instance;
}
} // namespace bridge
#include "TradingFormPanelHelper.h"
#include <QDateTime>
#include <cmath>
#include <algorithm>
namespace bridge {
TradingFormPanelHelper::TradingFormPanelHelper(QObject* parent)
    : QObject(parent) {}
//
// 模式判断
//
bool TradingFormPanelHelper::isEquityMode(const QString& mode) const {
    return mode == "stock" || mode == "margin_buy" || mode == "margin_sell";
}
bool TradingFormPanelHelper::isValidEquityCode(const QString& code) const {
    if (code.isEmpty()) return false;
    // 6位数字代码或交易所前缀+代码
    if (code.length() == 6) {
        bool ok = false;
        code.toLongLong(&ok);
        return ok;
    }
    return code.contains('.');
}
int TradingFormPanelHelper::priceDigitsForMode(const QString& mode) const {
    if (mode == "futures") return 1;
    if (mode == "options") return 4;
    return 2; // stock/margin
}
double TradingFormPanelHelper::priceStepForMode(const QString& mode) const {
    if (mode == "futures") return 1.0;
    if (mode == "options") return 0.0001;
}

```

```
    return 0.01;
}
double TradingFormPanelHelper::roundPriceByMode(const QString& mode, double price) const {
    double step = priceStepForMode(mode);
    return std::round(price / step) * step;
}
double TradingFormPanelHelper::referencePriceForMode(const QString& mode,
    const QVariantMap& snapshot) const {
    if (snapshot.contains("preClose")) return snapshot.value("preClose").toDouble();
    if (snapshot.contains("lastPrice")) return snapshot.value("lastPrice").toDouble();
    return snapshot.value("price").toDouble();
}
QString TradingFormPanelHelper::formatDisplayPrice(double price, int digits) const {
    return QString::number(price, 'f', digits);
}
QString TradingFormPanelHelper::formatAmountCompact(double amount) const {
    if (std::abs(amount) >= 1e8) {
        return QString::number(amount / 1e8, 'f', 2) + QStringLiteral("{Z}");
    }
    if (std::abs(amount) >= 1e4) {
        return QString::number(amount / 1e4, 'f', 2) + QStringLiteral("{W}");
    }
    return QString::number(amount, 'f', 2);
}
QString TradingFormPanelHelper::extractQuoteTime(const QVariantMap& snapshot) const {
    return snapshot.value("updatedAt").toString();
}
// -----
// 订单状态字符串规范化
// -----
QString TradingFormPanelHelper::canonicalOrderStatus(const QString& rawStatus) const {
    const QString s = rawStatus.toLower().trimmed();
    if (s == "pending" || s == "new" || s == "accepted") return "pending";
    if (s == "submitted" || s == "sent") return "submitted";
    if (s == "partial" || s == "partially_filled" || s == "partialfilled") return "partially_filled";
    if (s == "filled" || s == "done" || s == "executed") return "filled";
    if (s == "cancelled" || s == "canceled") return "cancelled";
    if (s == "rejected" || s == "refused") return "rejected";
    if (s == "expired" || s == "timeout") return "expired";
    if (s == "paused") return "paused";
    if (s == "checkpoint") return "checkpoint";
    return s.isEmpty() ? QStringLiteral("unknown") : s;
}
bool TradingFormPanelHelper::canCancelOrder(const QString& status) const {
    const QString s = canonicalOrderStatus(status);
    return s == "pending" || s == "submitted" || s == "partially_filled" || s == "paused";
}
bool TradingFormPanelHelper::canApproveCheckpoint(const QString& status) const {
    return canonicalOrderStatus(status) == "checkpoint";
}
```

```

bool TradingFormPanelHelper::canRetryCheckpoint(const QString& status) const {
    const QString s = canonicalOrderStatus(status);
    return s == "rejected" || s == "expired";
}
bool TradingFormPanelHelper::canResumePause(const QString& status) const {
    return canonicalOrderStatus(status) == "paused";
}
bool TradingFormPanelHelper::canRetryPause(const QString& status) const {
    const QString s = canonicalOrderStatus(status);
    return s == "rejected" || s == "expired";
}
QString TradingFormPanelHelper::checkpointActionLabel(const QString& status) const {
    return canApproveCheckpoint(status) ? QStringLiteral("批准检查点")
        : QStringLiteral("重试检查点");
}
QString TradingFormPanelHelper::pauseActionLabel(const QString& status) const {
    return canResumePause(status) ? QStringLiteral("恢复执行")
        : QStringLiteral("重试执行");
}
// =====
// Q_INVOKABLE 公共方法
// =====
QVariantMap TradingFormPanelHelper::buildHeaderState(const QString& mode,
    const QString& symbol,
    const QString& tabLabel,
    const QVariantMap& marketSnapshot) const {
    QVariantMap header;
    header["openingMarketWindow"] = false;
    // 参考价文本
    double preClose = marketSnapshot.value("preClose").toDouble();
    header["referenceText"] = preClose > 0.0
        ? QStringLiteral("#昨收 ") + formatDisplayPrice(preClose, priceDigitsForMode(mode))
        : QStringLiteral("--");
    // 模式显示标题
    if (mode == "stock") header["currentModeDisplayTitle"] = QStringLiteral("股票");
    else if (mode == "margin_buy") header["currentModeDisplayTitle"] = QStringLiteral("融资买入");
    else if (mode == "margin_sell") header["currentModeDisplayTitle"] = QStringLiteral("融券卖出");
    else if (mode == "futures") header["currentModeDisplayTitle"] = QStringLiteral("期货");
    else if (mode == "options") header["currentModeDisplayTitle"] = QStringLiteral("期权");
    else header["currentModeDisplayTitle"] = tabLabel;
    // 当前价格文本
    double price = marketSnapshot.value("price").toDouble();
    header["modePriceText"] = price > 0.0
        ? formatDisplayPrice(price, priceDigitsForMode(mode))
        : QStringLiteral("--");
    header["quoteTime"] = extractQuoteTime(marketSnapshot);
    return header;
}
QStringList TradingFormPanelHelper::quickButtonsForMode(const QString& mode) const {
    QStringList buttons;

```



```

    if (mode == "stock" || mode == "margin_buy" || mode == "margin_sell") {
        buttons << "100" << "200" << "500" << "1000" << "2000" << "5000" << "10000";
    } else if (mode == "futures") {
        buttons << "1" << "2" << "5" << "10" << "20" << "50";
    } else if (mode == "options") {
        buttons << "1" << "5" << "10" << "20" << "50" << "100";
    }
    return buttons;
}
}
QVariantList TradingFormPanelHelper::equityQuickPriceButtons() const {
    QVariantList buttons;
    auto add = [&](const QString& code, const QString& label) {
        QVariantMap btn;
        btn["code"] = code;
        btn["label"] = label;
        buttons.append(btn);
    };
    add("opponent", QStringLiteral("对手价"));
    add("bid1", QStringLiteral("买一"));
    add("ask1", QStringLiteral("卖一"));
    add("latest", QStringLiteral("最新价"));
    add("upper", QStringLiteral("涨停"));
    add("lower", QStringLiteral("跌停"));
    return buttons;
}
QVariantMap TradingFormPanelHelper::buildEquityDisplay(
    const QString& eqMode, const QString& currentMode,
    const QString& code, const QString& shares, const QString& priceType,
    const QString& price, const QVariantMap& marketSnapshot,
    const QVariantMap& depthSnapshot, double availableCapital,
    const QVariantMap& posSummary, bool posError) const {
    Q_UNUSED(code)
    Q_UNUSED(depthSnapshot)
    QVariantMap display;
    // 标的摘要
    QString symbol = marketSnapshot.value("symbol").toString();
    QString name = marketSnapshot.value("name").toString();
    display["identitySummary"] = symbol.isEmpty()
        ? QStringLiteral("--") : symbol + " " + name;
    display["identityColor"] = posError ? "danger" : "neutral";
    // 价格摘要
    double priceVal = price.toDouble();
    int digits = priceDigitsForMode(currentMode);
    QString priceDisplay = priceVal > 0.0
        ? formatDisplayPrice(priceVal, digits)
        : formatDisplayPrice(marketSnapshot.value("price").toDouble(), digits);
    display["priceSummary"] = priceDisplay;
    // 金额摘要
    double sharesVal = shares.toDouble();
    double amount = sharesVal * (priceVal > 0.0 ? priceVal

```

```
        : marketSnapshot.value("price").toDouble());
display["amountSummary"] = formatAmountCompact(amount);
display["availableCapital"] = availableCapital;
return display;
}
QVariantMap TradingFormPanelHelper::syncEquityReferenceState(
    const QString& mode, const QString& priceType, const QString& priceInput,
    double autoPrice, const QString& autoPriceType, const QVariantMap& marketSnapshot,
    const QVariantMap& depthSnapshot, bool openingMarketWindow) const {
    Q_UNUSED(depthSnapshot)
    QVariantMap state;
    state["priceType"] = priceType;
    state["priceInput"] = priceInput;
    state["autoPrice"] = autoPrice;
    state["autoPriceType"] = autoPriceType;
    if (openingMarketWindow && priceType != "limit") {
        state["priceType"] = QStringLiteral("limit");
        state["priceInput"] = formatDisplayPrice(
            referencePriceForMode(mode, marketSnapshot), priceDigitsForMode(mode));
        state["autoPrice"] = referencePriceForMode(mode, marketSnapshot);
        state["autoPriceType"] = QStringLiteral("latest");
    }
    return state;
}
QString TradingFormPanelHelper::formattedModePriceInput(const QString& targetMode,
    double numericPrice) const {
    return formatDisplayPrice(numericPrice, priceDigitsForMode(targetMode));
}
double TradingFormPanelHelper::adjustedModePriceInput(const QString& targetMode,
    const QString& priceInput,
    const QVariantMap& marketSnapshot,
    double stepDelta) const {
    double currentPrice = priceInput.toDouble();
    if (currentPrice <= 0.0) {
        currentPrice = marketSnapshot.value("price").toDouble();
    }
    if (currentPrice <= 0.0) return 0.0;
    double step = priceStepForMode(targetMode);
    double adjusted = currentPrice + stepDelta * step;
    return std::max(0.0, roundPriceByMode(targetMode, adjusted));
}
double TradingFormPanelHelper::resolveEquityShortcutPrice(
    const QString& shortcutCode, const QString& targetMode,
    const QVariantMap& marketSnapshot, const QVariantMap& depthSnapshot) const {
    Q_UNUSED(targetMode)
    if (shortcutCode == "latest" || shortcutCode == "last") {
        return marketSnapshot.value("price").toDouble();
    }
    if (shortcutCode == "opponent") {
        // 对手价：买用卖一， 卖用买一
    }
}
```

```

        return marketSnapshot.value("price").toDouble();
    }
    if (shortcutCode == "bid1") {
        QVariantList bids = depthSnapshot.value("bids").toList();
        return bids.isEmpty() ? 0.0 : bids.first().toMap().value("price").toDouble();
    }
    if (shortcutCode == "ask1") {
        QVariantList asks = depthSnapshot.value("asks").toList();
        return asks.isEmpty() ? 0.0 : asks.first().toMap().value("price").toDouble();
    }
    if (shortcutCode == "upper") {
        return marketSnapshot.value("upperLimit").toDouble();
    }
    if (shortcutCode == "lower") {
        return marketSnapshot.value("lowerLimit").toDouble();
    }
    return 0.0;
}

QString TradingFormPanelHelper::equityShortcutButtonText(
    const QString& code, const QString& label, const QString& mode,
    const QVariantMap& marketSnapshot, const QVariantMap& depthSnapshot) const {
    double price = resolveEquityShortcutPrice(code, mode, marketSnapshot, depthSnapshot);
    if (price > 0.0) {
        return label + "\n" + formatDisplayPrice(price, priceDigitsForMode(mode));
    }
    return label + "\n--";
}

QVariantMap TradingFormPanelHelper::buildOrderPresentation(
    const QVariantMap& orderData) const {
    QVariantMap pres;
    // rawStatus 是 mapServiceOrders 输出的原始状态字符串(大写), status 可能是已翻译的中文
    QString rawStatus = orderData.value("rawStatus").toString();
    if (rawStatus.isEmpty())
        rawStatus = orderData.value("status").toString();
    QString status = canonicalOrderStatus(rawStatus);
    pres["normalizedStatus"] = status;
    // 数量 — mapServiceOrders 输出字段为 qty, 兼容旧字段 quantity
    double quantity = orderData.value("qty").toDouble();
    if (quantity <= 0.0)
        quantity = orderData.value("quantity").toDouble();
    double price = orderData.value("price").toDouble();
    double amount = price * quantity;
    pres["headlineAmount"] = formatAmountCompact(amount);
    // 价格摘要
    pres["priceSummary"] = formatDisplayPrice(price, 2) +
        QStringLiteral(" × ") + QString::number(static_cast<qint64>(quantity));
    // 成交摘要 — mapServiceOrders 输出字段为 filledQty, 兼容 filledQuantity
    double filledQty = orderData.value("filledQty").toDouble();
    if (filledQty <= 0.0)
        filledQty = orderData.value("filledQuantity").toDouble();

```

```
pres["filledSummary"] = filledQty > 0.0
    ? QStringLiteral("已成交 ") + QString::number(static_cast<qint64>(filledQty))
    : QStringLiteral("等待成交");
// 辅助摘要
pres["auxiliarySummary"] = orderData.value("message").toString();
// 操作标志
pres["canCancel"] = canCancelOrder(status);
pres["canApproveManualCheckpoint"] = canApproveCheckpoint(status);
pres["canRetryManualCheckpoint"] = canRetryCheckpoint(status);
pres["checkpointActionLabel"] = checkpointActionLabel(status);
pres["canResumeExecutionPause"] = canResumePause(status);
pres["canRetryExecutionPause"] = canRetryPause(status);
pres["executionPauseActionLabel"] = pauseActionLabel(status);
return pres;
}
} // namespace bridge
#include "TradingConnectionConfigService.h"
#include "ConfigVariantAdapter.h"
#include <QCoreApplication>
#include <QDir>
#include <QFileInfo>
#include <QJsonArray>
#include <QMutexLocker>
#include <QTimer>
#include "foundation/config/ConfigManager.hpp"
#include "foundation/log/logging.hpp"
namespace bridge {
TradingConnectionConfigService::TradingConnectionConfigService(QObject* parent)
: QObject(parent) {
    m_configFilePath = resolveConfigFilePath();
    m_currentConfig = readConfigFile();
    m_initialized = true;
    INTERNAL_DEBUG_STREAM << "[TradingConnectionConfig] 已创建, boundStrategyId ="
        << m_currentConfig.value("boundStrategyId").toString().toStdString();
}
QVariantMap TradingConnectionConfigService::currentConfiguration() const {
    QMutexLocker lock(&m_mutex);
    return m_currentConfig;
}
QString TradingConnectionConfigService::configFilePath() const {
    return m_configFilePath;
}
// =====
// 配置持久化
// =====
QVariantMap TradingConnectionConfigService::loadConfiguration() {
    QMutexLocker lock(&m_mutex);
    m_currentConfig = readConfigFile();
    m_initialized = true;
    INTERNAL_DEBUG_STREAM << "[TradingConnectionConfig] 已加载配置:"
```

```

        << "\n boundStrategyId =" << m_currentConfig.value("boundStrategyId").toString().toString()
        << "\n gmStrategyId =" << m_currentConfig.value("gmStrategyId").toString().toString()
        << "\n strategyId =" << m_currentConfig.value("strategyId").toString().toString()
        << "\n runtimeStrategyId =" << m_currentConfig.value("runtimeStrategyId").toString().toString()
        << "\n accountId =" << m_currentConfig.value("accountId").toString().toString()
        << "\n token =" << (m_currentConfig.value("token").toString().left(8) + "...").toString()
        << "\n boundStrategies count =" << m_currentConfig.value("boundStrategies").toList().size();
    emit currentConfigurationChanged();
    return m_currentConfig;
}
bool TradingConnectionConfigService::saveConfiguration(const QVariantMap& payload) {
    QMutexLocker lock(&m_mutex);
    QVariantMap normalized = normalizeConfiguration(payload);
    m_currentConfig = normalized;
    if (writeConfigFile(normalized)) {
        emit currentConfigurationChanged();
        return true;
    }
    emit errorOccurred(QStringLiteral("保存交易配置失败"));
    return false;
}
QVariantMap TradingConnectionConfigService::defaultConfiguration() const {
    QVariantMap cfg;
    cfg["enabled"] = false;
    cfg["readOnly"] = false;
    cfg["boundStrategyId"] = QString();
    cfg["boundStrategyName"] = QString();
    cfg["runtimeStrategyId"] = QString();
    cfg["accountId"] = QString();
    cfg["liveAccountId"] = QString();
    cfg["simAccountId"] = QString();
    cfg["gmStrategyId"] = QString();
    cfg["token"] = QString();
    cfg["serverUrl"] = QString();
    cfg["accountProfile"] = QStringLiteral("simulation");
    cfg["boundStrategies"] = QVariantList();
    cfg["runtimeRuleDefaults"] = QVariantMap();
    cfg["symbols"] = QVariantList();
    cfg["updatedAt"] = QString();
    cfg["version"] = 1;
    // —— 设置页: 交易执行 ——
    cfg["defaultOrderType"] = QStringLiteral("Limit");
    cfg["eodTriggerTime"] = QStringLiteral("14:50");
    cfg["compensationEndTime"] = QStringLiteral("09:30");
    cfg["triggerWindowMinutes"] = 10;
    // 收盘 EOD 回调兜底窗口 (收盘时段 tick 触发, 时间由配置决定, 零硬编码零兜底)
    cfg["eodCallbackStartTime"] = QStringLiteral("14:50");
    cfg["eodCallbackEndTime"] = QStringLiteral("15:00");
    cfg["useBoardLot"] = true;
    cfg["limitPriceTolerance"] = 9.9;
}

```

```
// —— 设置页: 同步调度 (封锁结束不单独配置, 由 eodTriggerTime 派生) ——
cfg["syncTriggerTime"] = QStringLiteral("15:01");
cfg["syncBlockStart"] = QStringLiteral("09:25");
cfg["syncDailyBatchSize"] = 100;
cfg["syncMinuteBatchSize"] = 50;
cfg["syncDbFlushSize"] = 500;
cfg["syncCoverageRatio"] = 0.9;
cfg["syncAdjRetryCount"] = 2;
return cfg;
}
//
// 策略绑定管理
//
QVariantMap TradingConnectionConfigService::bindStrategyConfiguration(
    const QString& strategyId, const QString& strategyName,
    bool isPrimary, bool autoStart) {
    QMutexLocker lock(&m_mutex);
    if (!m_initialized) {
        m_currentConfig = readConfigFile();
        m_initialized = true;
    }
    QString sid = sanitizeStrategyId(strategyId);
    QVariantMap entry;
    entry["strategyId"] = sid;
    entry["strategyName"] = strategyName;
    entry["isPrimary"] = isPrimary;
    entry["autoStart"] = autoStart;
    entry["enabled"] = true;
    entry["readOnly"] = false;
    if (isPrimary) {
        m_currentConfig["boundStrategyId"] = sid;
        m_currentConfig["boundStrategyName"] = strategyName;
        m_currentConfig["runtimeStrategyId"] = sid;
    }
    mergeBoundStrategy(entry);
    writeConfigFile(m_currentConfig);
    QVariantMap result;
    result["success"] = true;
    result["message"] = QStringLiteral("策略绑定成功");
    result["enabled"] = true;
    result["readOnly"] = false;
    result["readyForTrading"] = true;
    QVariantMap startupGate = evaluateStartupGate(false);
    result["startupGate"] = startupGate;
    emit currentConfigurationChanged();
    return result;
}
QVariantMap TradingConnectionConfigService::addBoundStrategyConfiguration(
    const QString& strategyId, const QString& strategyName,
    bool isPrimary, bool autoStart) {
```

```
    return bindStrategyConfiguration(strategyId, strategyName, isPrimary, autoStart);
}
QVariantMap TradingConnectionConfigService::removeBoundStrategyConfiguration(
    const QString& strategyId) {
    QMutexLocker lock(&m_mutex);
    if (!m_initialized) {
        m_currentConfig = readConfigFile();
        m_initialized = true;
    }
    QString sid = sanitizeStrategyId(strategyId);
    bool removed = removeBoundStrategy(sid);
    // 如果移除的是当前主策略, 清除相关引用
    if (m_currentConfig.value("boundStrategyId").toString() == sid) {
        m_currentConfig["boundStrategyId"] = QString();
        m_currentConfig["boundStrategyName"] = QString();
    }
    if (m_currentConfig.value("runtimeStrategyId").toString() == sid) {
        m_currentConfig["runtimeStrategyId"] = QString();
    }
    writeConfigFile(m_currentConfig);
    emit currentConfigurationChanged();
    QVariantMap result;
    result["success"] = removed;
    result["message"] = removed ? QStringLiteral("策略解绑成功")
        : QStringLiteral("未找到要解绑的策略");
    return result;
}
// =====
// 启动风控
// =====
QVariantMap TradingConnectionConfigService::evaluateStartupGate(bool forceRecheck) {
    Q_UNUSED(forceRecheck)
    QMutexLocker lock(&m_mutex);
    if (!m_initialized) {
        m_currentConfig = readConfigFile();
        m_initialized = true;
    }
    QStringList reasons;
    bool ready = true;
    // 检查是否有绑定策略
    QVariantList strategies = boundStrategies();
    if (strategies.isEmpty()) {
        ready = false;
        reasons << QStringLiteral("未绑定任何策略");
    }
    // 检查是否有 accountId 配置
    QString accountId = m_currentConfig.value("accountId").toString();
    if (accountId.isEmpty()) {
        QString simId = m_currentConfig.value("simAccountId").toString();
        QString liveId = m_currentConfig.value("liveAccountId").toString();
    }
}
```

```
        if (simId.isEmpty() && liveId.isEmpty()) {
            ready = false;
            reasons << QStringLiteral("未配置交易账户");
        }
    }
    return buildStartupGateResult(ready,
        ready ? QStringLiteral("OK") : QStringLiteral("CONFIG_INCOMPLETE"),
        reasons);
}
// =====
// 客户端进程状态
// =====
QVariantMap TradingConnectionConfigService::refreshClientProcessStatus() {
    return detectClientProcesses();
}
void TradingConnectionConfigService::refreshClientProcessStatusAsync() {
    QTimer::singleShot(0, this, [this]() {
        QVariantMap status = detectClientProcesses();
        emit clientProcessStatusChanged(status);
    });
}
// =====
// 内部辅助方法
// =====
QString TradingConnectionConfigService::resolveConfigFilePath() const {
    return QString::fromStdString(
        foundation::config::ConfigManager::instance()
            .configFilePath(foundation::config::ConfigFile::TradingConnection));
}
QVariantMap TradingConnectionConfigService::normalizeConfiguration(const QVariantMap& raw) const {
    QVariantMap normalized;
    // 键名规范化：统一使用 camelCase
    auto normalizeKey = [](const QString& key) -> QString {
        if (key == "accountId" || key == "liveAccountId" || key == "simAccountId"
            || key == "gmStrategyId" || key == "strategy_id" || key == "gm_strategy_id")
            return key;
        // 将下划线键转为驼峰
        if (key.contains('_')) {
            QString result;
            bool capitalize = false;
            for (QChar c : key) {
                if (c == '_') { capitalize = true; continue; }
                result += capitalize ? c.toUpper() : c;
                capitalize = false;
            }
            return result;
        }
        return key;
    };
    for (auto it = raw.begin(); it != raw.end(); ++it) {
```



```

        normalized[normalizeKey(it.key())] = it.value();
    }
    // accountId 别名合并 (不引入默认值)
    if (!normalized.contains("accountId") || normalized.value("accountId").toString().isEmpty()) {
        QString simId = normalized.value("simAccountId").toString();
        if (!simId.isEmpty()) {
            normalized["accountId"] = simId;
        } else if (normalized.contains("liveAccountId")) {
            normalized["accountId"] = normalized.value("liveAccountId").toString();
        }
    }
    normalized["updatedAt"] = QDateTime::currentDateTime().toString(Qt::ISODate);
    return normalized;
}

bool TradingConnectionConfigService::writeConfigFile(const QVariantMap& config) const {
    auto node = toConfigNode(config);
    return foundation::config::ConfigManager::instance()
        .saveConfigFile(foundation::config::ConfigFile::TradingConnection, node);
}

QVariantMap TradingConnectionConfigService::readConfigFile() const {
    auto cfg = foundation::config::ConfigManager::instance()
        .loadConfigFile(foundation::config::ConfigFile::TradingConnection);
    if (!cfg || cfg->isNull()) return QVariantMap();
    return normalizeConfiguration(toVariantMap(*cfg));
}

QVariantMap TradingConnectionConfigService::buildStartupGateResult(
    bool ready, const QString& reasonCode, const QStringList& reasons) const {
    QVariantMap gate;
    gate["ready"] = ready;
    gate["reasonCode"] = reasonCode;
    gate["reasons"] = reasons;
    return gate;
}

QVariantMap TradingConnectionConfigService::detectClientProcesses() const {
    QVariantMap status;
    status["running"] = false;
    status["processNames"] = QVariantList();
    status["clientProcessRunning"] = false;
    status["clientProcessStatus"] = QStringLiteral("未检测");
    // MVP: 不做实际的进程检测 (原实现依赖 tasklist 命令和特定进程名)
    return status;
}

QString TradingConnectionConfigService::sanitizeStrategyId(const QString& raw) const {
    QString sanitized = raw.trimmed().toLower();
    // 只保留字母数字和下划线
    QString result;
    for (QChar c : sanitized) {
        if (c.isLetterOrNumber() || c == '_' || c == '-') {
            result += c;
        }
    }
}

```

```
}
// 限制长度
if (result.length() > 64) result = result.left(64);
return result.isEmpty() ? raw : result;
}
QVariantList TradingConnectionConfigService::boundStrategies() const {
    return m_currentConfig.value("boundStrategies").toList();
}
void TradingConnectionConfigService::setBoundStrategies(const QVariantList& strategies) {
    m_currentConfig["boundStrategies"] = strategies;
}
void TradingConnectionConfigService::mergeBoundStrategy(const QVariantMap& entry) {
    QVariantList strategies = boundStrategies();
    QString sid = entry.value("strategyId").toString();
    // 更新已有条目或追加
    for (int i = 0; i < strategies.size(); ++i) {
        QVariantMap item = strategies[i].toMap();
        if (item.value("strategyId").toString() == sid) {
            strategies[i] = entry;
            setBoundStrategies(strategies);
            return;
        }
    }
    strategies.append(entry);
    setBoundStrategies(strategies);
}
bool TradingConnectionConfigService::removeBoundStrategy(const QString& strategyId) {
    QVariantList strategies = boundStrategies();
    for (int i = 0; i < strategies.size(); ++i) {
        if (strategies[i].toMap().value("strategyId").toString() == strategyId) {
            strategies.removeAt(i);
            setBoundStrategies(strategies);
            return true;
        }
    }
    return false;
}
TradingConnectionConfigService* TradingConnectionConfigService::instance() {
    static TradingConnectionConfigService s_instance;
    return &s_instance;
}
} // namespace bridge
using SA = domain::strategy::SpecialAction;
SA action = domain::strategy::RiskEvaluator::specialActionFromString(actionRaw);
if (action == SA::CashRepay) {
    order.setActionKind(domain::trading::ActionKind::CashRepay);
    order.setBoardLotMode(false);
} else if (action == SA::ShareReturn) {
    order.setActionKind(domain::trading::ActionKind::ShareReturn);
    order.setBoardLotMode(false);
}
```

```

    }
}
// —— 可选字段 ——
if (orderMap.contains("signalStrength"))
    order.setSignalStrength(orderMap.value("signalStrength").toDouble());
if (orderMap.contains("cashAmount"))
    order.setCashAmount(orderMap.value("cashAmount").toDouble());
if (orderMap.contains("clientOrderId"))
    order.setClientId(orderMap.value("clientOrderId").toString().toStdString());
emit orderRequested(orderMap);
emit orderRequestPublished(orderMap);
// 构建 OrderRequest (风控在 Trading 引擎 submitOrder 内部统一完成)
engine::OrderRequest engineReq;
auto& accEng = engine::AccountEngine::instance();
auto account = accEng.account();
domain::trading::OrderBuilder manualBuilder;
bool isBuy = (order.side() == domain::strategy::OrderDirection::Buy);
auto pe = isBuy ? domain::trading::PositionEffect::Open
               : domain::trading::PositionEffect::Close;
engineReq = manualBuilder.buildManualOrder(
    order.symbol(),
    isBuy ? engine::OrderSide::Buy : engine::OrderSide::Sell,
    order.price(), order.quantity(), pe,
    strategyId.toStdString(), account.accountId());
if (!engineReq.isValid()) {
    QVariantMap out;
    out["accepted"] = false;
    out["message"] = QStringLiteral("订单构建失败: 价格/数量无效");
    setLastError(out["message"].toString());
    return out;
}
// 篮子ID — 手动单也标记以便审计追溯
{
    static std::atomic<uint64_t> s_manualBasketSeq{0};
    engineReq.setExtension(domain::trading::ExtKey::kBasketId,
        s_manualBasketSeq.fetch_add(1));
}
// TradeOrder 字段对齐 OrderBuilder 输出
order.setClOrdId(engineReq.clOrdId());
order.setAccountId(engineReq.accountId());
order.setCurrency(engineReq.currency());
order.setExchange(engineReq.exchange());
order.setPositionEffect(static_cast<domain::strategy::PositionEffect>(engineReq.positionEffect()));
auto result = domain::trading::TradeExecutionEngine::instance().submitOrder(order);
// —— 记录结果 ——
QVariantMap out;
out["accepted"] = result.succeeded();
out["message"] = QString::fromStdString(result.message());
if (!result.succeeded()) {
    setLastError(QString::fromStdString(result.message()));
}

```

```

}
// —— 引擎提前拒绝时(校验/调度/风控/未初始化), 回调未触发, 桥接层负责写入 + 通知 ——
// 成功路径上 TradeExecutionEngine::submitOrder() 内部已通过 setOnOrderAccepted
// 回调完成 appendRecentOrder + 信号发射, 桥接层不重复处理。
if (!result.brokerOrderId().valid()) {
    QVariantMap recentEntry;
    recentEntry[kBrokerOrderId] = QString::fromStdString(result.brokerOrderId().text());
    recentEntry[kSymbol] = orderMap.value(kSymbol);
    recentEntry[kSide] = orderMap.value(kSide);
    recentEntry[kPrice] = orderMap.value(kPrice);
    recentEntry[kQuantity] = orderMap.value(kQuantity);
    recentEntry[kStatus] = QStringLiteral("REJECTED");
    recentEntry[kRawStatus] = QStringLiteral("REJECTED");
    recentEntry["message"] = QString::fromStdString(result.message());
    recentEntry["submittedAt"] = QDateTime::currentDateTime().toString(Qt::ISODate);
    appendRecentOrder(recentEntry);
    emit orderStatusChanged(recentEntry);
    emit orderStatusPublished(recentEntry);
}
return out;
}
bool TradeExecutionBridge::submitBridgeOrder(const QVariantMap& request) {
    QVariantMap result = submitOrder(request);
    return result.value("accepted").toBool();
}
void TradeExecutionBridge::publishOrderStatusUpdate(const QString& orderId,
                                                    const QString& status,
                                                    const QString& message) {
    QVariantMap capturedItem;
    bool foundItem = false;
    {
        QMutexLocker lock(&m_recentOrdersMutex);
        for (int i = 0; i < m_recentOrders.size(); ++i) {
            QVariantMap item = m_recentOrders[i].toMap();
            if (item.value(kBrokerOrderId).toString() == orderId
                || item.value("clientOrderId").toString() == orderId
                || item.value("id").toString() == orderId) {
                item[kStatus] = status;
                item[kRawStatus] = status;
                if (!message.isEmpty())
                    item["message"] = message;
                m_recentOrders[i] = item;
                capturedItem = item;
                foundItem = true;
                break;
            }
        }
    }
} // 锁释放
if (foundItem) {
    emit orderStatusChanged(capturedItem);
}

```

```
        emit orderStatusPublished(capturedItem);
        emit recentOrdersChanged();
    }
}

bool TradeExecutionBridge::cancelOrder(const QString& brokerOrderId) {
    ensureInitialized();
    if (!m_initialized) return false;
    domain::trading::BrokerOrderId id(brokerOrderId.toStdString());
    auto* engine = &domain::trading::TradeExecutionEngine::instance();
    if (!engine) {
        setLastError(QStringLiteral("交易引擎未初始化"));
        return false;
    }
    bool ok = engine->cancelOrder(id);
    if (ok)
        publishOrderStatusUpdate(brokerOrderId, QStringLiteral("PENDING_CANCEL"), QStringLiteral("撤单已提交"));
    else
        setLastError(QStringLiteral("撤单失败"));
    return ok;
}

bool TradeExecutionBridge::resumeExecutionPause(const QString& executionScopeId,
                                                const QString& pausedBatchId) {
    auto* engine = &domain::trading::TradeExecutionEngine::instance();
    if (!engine) {
        setLastError(QStringLiteral("交易引擎未初始化"));
        return false;
    }
    bool ok = engine->resumeExecutionPause(executionScopeId.toStdString(),
                                           pausedBatchId.toStdString());
    if (!ok) setLastError(QStringLiteral("恢复执行暂停失败"));
    return ok;
}

bool TradeExecutionBridge::approveExecutionCheckpoint(const QString& executionScopeId,
                                                      const QString& batchId) {
    auto* engine = &domain::trading::TradeExecutionEngine::instance();
    if (!engine) {
        setLastError(QStringLiteral("交易引擎未初始化"));
        return false;
    }
    bool ok = engine->approveExecutionCheckpoint(executionScopeId.toStdString(),
                                                batchId.toStdString());
    if (!ok) setLastError(QStringLiteral("批准执行检查点失败"));
    return ok;
}

bool TradeExecutionBridge::cancelManualTestOrder(const QString& orderId) {
    domain::trading::BrokerOrderId id(orderId.toStdString());
    auto* engine = &domain::trading::TradeExecutionEngine::instance();
    if (!engine) {
        setLastError(QStringLiteral("交易引擎未初始化"));
        return false;
    }
}
```

```

    }
    bool ok = engine->cancelOrder(id);
    if (ok)
        publishOrderStatusUpdate(orderId, QStringLiteral("PENDING_CANCEL"), QStringLiteral("撤单已提交"));
    else
        setLastError(QStringLiteral("取消手动测试订单失败"));
    return ok;
}
QVariantMap TradeExecutionBridge::quickClosePosition(const QString& symbol, const QString& mode) {
    QVariantMap out;
    out["accepted"] = false;
    ensureInitialized();
    // 1. 查持仓 — 只做最基本的判空
    auto positions = engine::AccountEngine::instance().positions();
    QString searchSym = symbol.trimmed().toUpper();
    const engine::Position* found = nullptr;
    for (auto& p : positions) {
        if (QString::fromStdString(p.symbol).trimmed().toUpper() == searchSym) {
            found = &p; break;
        }
    }
    if (!found || found->quantity <= 0) {
        out["message"] = QStringLiteral("未找到持仓") + searchSym;
        return out;
    }
    QString m = mode.trimmed().toLower();
    QString side = (found->quantity >= 0) ? QStringLiteral("SELL") : QStringLiteral("BUY");
    std::int64_t closeQty = found->availableQty > 0 ? found->availableQty : found->quantity;
    // 2. 价格 — 跌停价卖出, 取不到用持仓成本
    double price = 0;
    auto quote = engine::GmSessionEngine::instance().fetchQuote(found->symbol);
    if (quote && quote->valid && quote->preClose > 0) {
        double preClose = quote->preClose;
        double limitRatio = domain::trading::boardLimitRatio(found->symbol);
        price = (side == "SELL") ? (preClose * (1.0 - limitRatio))
            : (preClose * (1.0 + limitRatio));
        price = domain::trading::roundPriceForBoard(price, m.toStdString());
    }
    if (price <= 0) price = found->lastPrice > 0 ? found->lastPrice : found->costPrice;
    if (price <= 0) price = 0.01;
    // 3. 直连网关, 不做本地风控/T+1/数量校验, 全部由网关裁决
    auto* strategy = engine::GmSessionEngine::instance().strategy();
    if (!strategy) {
        out["message"] = QStringLiteral("交易会话未就绪");
        return out;
    }
    std::string gmSym = foundation::market::AStockSymbol::fromString(found->symbol).gmSymbol();
    if (gmSym.empty()) {
        out["message"] = QStringLiteral("无效代码:") + searchSym;
        return out;
    }
}

```

```

}
int gmSide = (side == "BUY") ? 1 : 2;
Order gmOrder = strategy->place_order(gmSym.c_str(),
    static_cast<int>(closeQty),
    gmSide, 1 /*Limit*/,
    2 /*平仓*/, price, 0, 0, 0.0, 0, NULL);
bool accepted = (gmOrder.cl_ord_id[0] != '\0');
std::string brokerId = accepted ? gmOrder.cl_ord_id : "";
// 4. 网关原始错误
std::string gwError;
if (!accepted) {
    auto err = strategy->get_last_error_detail();
    gwError = (err && err[0]) ? err : "place_order rejected";
}
// 5. 记录到 recentOrders
QVariantMap recentEntry;
recentEntry[kBrokerOrderId] = QString::fromStdString(brokerId);
recentEntry[kSymbol] = searchSym;
recentEntry[kSide] = side;
recentEntry[kPrice] = price;
recentEntry[kQuantity] = static_cast<double>(closeQty);
recentEntry[kStatus] = accepted ? QStringLiteral("SUBMITTED") : QStringLiteral("REJECTED");
recentEntry[kRawStatus] = recentEntry[kStatus];
recentEntry["message"] = accepted
    ? QStringLiteral("平仓已提交") : QString::fromStdString(gwError);
recentEntry["submittedAt"] = QDateTime::currentDateTime().toString(Qt::ISODate);
appendRecentOrder(recentEntry);
emit orderStatusChanged(recentEntry);
emit orderStatusPublished(recentEntry);
// 6. 注册到引擎追踪成交状态
if (accepted) {
    domain::trading::TradeOrder trackOrder;
    trackOrder.setSymbol(found->symbol);
    trackOrder.setBrokerOrderId(brokerId);
    trackOrder.setQuantity(closeQty);
    trackOrder.setPrice(price);
    trackOrder.setSide(side == "BUY" ? domain::strategy::OrderDirection::Buy
        : domain::strategy::OrderDirection::Sell);
    trackOrder.setStatus(domain::trading::OrderStatusValue::New);
    trackOrder.setStatusMessage("accepted");
    domain::trading::TradeExecutionEngine::instance().registerOrder(trackOrder);
}
out["accepted"] = accepted;
out[kStatus] = recentEntry[kStatus];
out["message"] = recentEntry["message"];
if (!gwError.empty())
    out["gwError"] = QString::fromStdString(gwError);
return out;
}
void TradeExecutionBridge::clearRecentOrders() {

```

```

{
    QMutexLocker lock(&m_recentOrdersMutex);
    m_recentOrders.clear();
}
emit recentOrdersChanged();
}
// —— Domain 工具方法 (薄转发到 OrderUtils.h) ——
QString TradeExecutionBridge::resolveLiveOrderType(
    const QString& rawType, const QString& optionType,
    const QString& underlying, const QString& exchange) const {
    return QString::fromUtf8(domain::trading::resolveLiveOrderType(
        rawType.toStdString(), optionType.toStdString(),
        underlying.toStdString(), exchange.toStdString()));
}
QString TradeExecutionBridge::resolveLiveOrderAction(
    const QString& rawType, const QString& side,
    const QString& positionEffect, const QString& action) const {
    return QString::fromUtf8(domain::trading::resolveLiveOrderAction(
        rawType.toStdString(), side.toStdString(),
        positionEffect.toStdString(), action.toStdString()));
}
QString TradeExecutionBridge::translateOrderSide(const QString& side) const {
    return QString::fromUtf8(domain::trading::translateOrderSide(side.toStdString()));
}
bool TradeExecutionBridge::isFuturesExchange(const QString& exchange) const {
    return domain::trading::isFuturesExchange(exchange.toStdString());
}
void TradeExecutionBridge::appendRecentOrder(const QVariantMap& order) {
{
    QMutexLocker lock(&m_recentOrdersMutex);
    m_recentOrders.prepend(order);
    if (m_recentOrders.size() > 50) m_recentOrders.removeLast();
}
    emit recentOrdersChanged();
}
void TradeExecutionBridge::setLastError(const QString& message) {
    if (m_lastErrorMessage == message) return;
    m_lastErrorMessage = message;
    emit lastErrorMessageChanged();
}
//
// PositionAccountBridge
//
PositionAccountBridge::PositionAccountBridge(QObject* parent)
    : QObject(parent) {}
bool PositionAccountBridge::initialized() const { return m_initialized; }
void PositionAccountBridge::initialize() {
    if (m_initialized) return;
    m_initialized = true;
    // 订阅 AccountEngine 数据变更 — 成交后 GmSdk 回调 → AccountEngine → 触发 refresh

```



```

    engine::AccountEngine::instance().addOnDataChanged([this]() {
        refresh();
    });
    emit initializedChanged();
    refresh();
}
void PositionAccountBridge::requestInitialSnapshot() {
    if (!m_initialized) initialize();
    refresh();
}
bool PositionAccountBridge::initialSnapshotLoaded() const {
    return m_initialized;
}
void PositionAccountBridge::refresh() {
    emit accountSnapshotChanged();
    emit positionsChanged();
    emit dataChanged();
}
QVariantMap PositionAccountBridge::accountSnapshot() const {
    const_cast<PositionAccountBridge*>(this)->initialize();
    QVariantMap map;
    if (!m_initialized) return map;
    auto snap = engine::AccountEngine::instance().snapshot();
    auto& acc = snap.account;
    map["totalAsset"] = acc.totalAsset;
    map["marketValue"] = acc.marketValue;
    map["availableCash"] = acc.availableCash;
    map["realizedPnl"] = acc.realizedPnl;
    // 浮动盈亏先用 gmsdk 推送值, 否则汇总各持仓
    double unrealized = acc.unrealizedPnl;
    if (unrealized == 0.0) {
        for (const auto& p : snap.positions)
            unrealized += p.unrealizedPnl;
    }
    map["unrealizedPnl"] = unrealized;
    map[kAccountId] = QString::fromStdString(acc.accountId);
    return map;
}
QVariantList PositionAccountBridge::positions() const {
    const_cast<PositionAccountBridge*>(this)->initialize();
    QVariantList list;
    if (!m_initialized) return list;
    for (auto& p : engine::AccountEngine::instance().positions()) {
        QVariantMap item;
        const QString sym = QString::fromStdString(p.symbol);
        item[kSymbol] = sym;
        item["name"] = StockNameResolver::name(sym);
        item[kSide] = p.quantity >= 0 ? "LONG" : "SHORT";
        item["type"] = "stock";
        item[kQuantity] = static_cast<double>(p.quantity);
    }
}

```

```

        item["availableQuantity"] = static_cast<double>(p.availableQty);
        item["closeableQuantity"] = static_cast<double>(p.availableQty);
        item["costBasis"]        = p.costPrice;
        item["kLastPrice"]       = p.lastPrice;
        item["marketValue"]      = p.marketValue;
        item["unrealizedPnl"]    = p.unrealizedPnl;
        list.append(item);
    }
    return list;
}
QVariantList PositionAccountBridge::recentOrderStatuses() const {
    QMutexLocker lock(&m_orderStatusesMutex);
    return m_recentOrderStatuses;
}
void PositionAccountBridge::appendOrderStatus(const QVariantMap& status) {
    bool updated = false;
    {
        QMutexLocker lock(&m_orderStatusesMutex);
        // 去重：按 brokerOrderId 更新已有条目
        const QString id = status.value(kBrokerOrderId).toString();
        for (int i = 0; i < m_recentOrderStatuses.size(); ++i) {
            QVariantMap item = m_recentOrderStatuses[i].toMap();
            if (item.value(kBrokerOrderId).toString() == id) {
                m_recentOrderStatuses[i] = status;
                updated = true;
                break;
            }
        }
    }
    if (!updated) {
        m_recentOrderStatuses.prepend(status);
        if (m_recentOrderStatuses.size() > 50) m_recentOrderStatuses.removeLast();
    }
    emit recentOrderStatusesChanged();
}
// —— Domain 工具方法 (薄转发到 PositionUtils.h 自由函数) ——
double PositionAccountBridge::normalizePositionQuantity(double rawQty, const QString& type) const {
    return static_cast<double>(
        domain::trading::normalizePositionQuantity(rawQty, type.toStdString()));
}
double PositionAccountBridge::calculatePositionMarketValue(const QVariantList& rawPositions) const {
    double total = 0.0;
    for (const auto& item : rawPositions) {
        total += item.toMap().value("marketValue", 0.0).toDouble();
    }
    return total;
}
QString PositionAccountBridge::positionTypeTitle(const QString& type) const {
    return QString::fromUtf8(domain::trading::positionTypeTitle(type.toStdString()));
}
}

```

```
QString PositionAccountBridge::positionUnit(const QString& type) const {
    return QString::fromUtf8(domain::trading::positionUnit(type.toString()));
}
QString PositionAccountBridge::positionSideLabel(const QString& side) const {
    return QString::fromUtf8(domain::trading::positionSideLabel(side.toString()));
}
QString PositionAccountBridge::closeableLabel(const QString& type, const QString& side) const {
    return QString::fromUtf8(domain::trading::closeableLabel(type.toString(), side.toString()));
}
// =====
// RiskControlBridge
// =====
RiskControlBridge::RiskControlBridge(QObject* parent)
    : QObject(parent) {
    m_timer.setInterval(3000);
    connect(&m_timer, &QTimer::timeout, this, &RiskControlBridge::refresh);
}
double RiskControlBridge::varUsagePercent() const { return m_varUsagePct; }
double RiskControlBridge::currentDrawdownPercent() const { return m_drawdownPct; }
double RiskControlBridge::currentTotalExposurePercent() const { return m_exposurePct; }
double RiskControlBridge::varBudgetAmount() const { return m_varBudget; }
double RiskControlBridge::estimatedVarAmount() const { return m_estimatedVar; }
void RiskControlBridge::initializeAsync() {
    // 不在此处调 TradingSystem::initialize() —— gateway 可能还没设置
    refresh();
    m_timer.start();
}
void RiskControlBridge::initialize() {
    initializeAsync();
}
void RiskControlBridge::refresh() {
    if (!engine::AccountEngine::instance().initialized()) return;
    auto acc = engine::AccountEngine::instance().account();
    double totalAsset = acc.totalAsset;
    double marketValue = acc.marketValue;
    bool changed = false;
    auto check = [&](double& oldVal, double newVal, double eps = 0.01) {
        if (std::abs(oldVal - newVal) > eps) { oldVal = newVal; changed = true; }
    };
    double exposurePct = totalAsset > 0.0 ? (marketValue / totalAsset) * 100.0 : 0.0;
    double varUsage = domain::strategy::RiskManager::estimateVar(exposurePct);
    check(m_exposurePct, exposurePct);
    check(m_varUsagePct, varUsage);
    if (changed) {
        emit varUsagePercentChanged();
        emit currentDrawdownPercentChanged();
        emit currentTotalExposurePercentChanged();
        emit varBudgetAmountChanged();
        emit estimatedVarAmountChanged();
    }
}
```

```
}
QVariantMap RiskControlBridge::buildPortfolioSnapshot(const QVariantMap& strategy,
const QVariantMap& backtestRecord) {
    QVariantMap snapshot;
    snapshot["timestamp"] = QDateTime::currentDateTime().toString(Qt::ISODate);
    snapshot[kStrategyId] = strategy.value(kStrategyId);
    if (!engine::AccountEngine::instance().initialized()) {
        snapshot[kStatus] = "unavailable";
        snapshot["positions"] = QVariantList();
        snapshot["diagnostics"] = QVariantMap();
        return snapshot;
    }
    auto snap = engine::AccountEngine::instance().snapshot();
    auto& acc = snap.account;
    snapshot["totalAsset"] = acc.totalAsset;
    snapshot["marketValue"] = acc.marketValue;
    snapshot["availableCash"] = acc.availableCash;
    snapshot["realizedPnl"] = acc.realizedPnl;
    snapshot["unrealizedPnl"] = acc.unrealizedPnl;
    snapshot[kStatus] = "ok";
    auto& positions = snap.positions;
    QVariantList posList;
    for (auto& p : positions) {
        QVariantMap item;
        const QString sym = QString::fromStdString(p.symbol);
        item[kSymbol] = sym;
        item["name"] = StockNameResolver::name(sym);
        item[kSide] = p.quantity >= 0 ? "LONG" : "SHORT";
        item["type"] = "stock";
        item[kQuantity] = static_cast<double>(p.quantity);
        item["costBasis"] = p.costPrice;
        item[kLastPrice] = p.lastPrice;
        item["marketValue"] = p.marketValue;
        item["unrealizedPnl"] = p.unrealizedPnl;
        posList.append(item);
    }
    snapshot["positions"] = posList;
    // 回测上下文诊断 (简化)
    QVariantMap diagnostics;
    diagnostics["backtestRecordId"] = backtestRecord.value("recordId");
    diagnostics["exposurePercent"] = m_exposurePct;
    diagnostics["varUsagePercent"] = m_varUsagePct;
    diagnostics["drawdownPercent"] = m_drawdownPct;
    snapshot["diagnostics"] = diagnostics;
    return snapshot;
}
// _____
// 掘金实盘网关 (bridge 层, 有 Qt)
// _____
} // namespace bridge
```